

# HARBOR BRIDGE

# TERMINAL PLATFORM

## Technical Expository Note

Architecture | RAMWS v2 | Security | Cubby Mobile Integration | Verification |  
Performance Backtesting | Qnode Fleet Audit

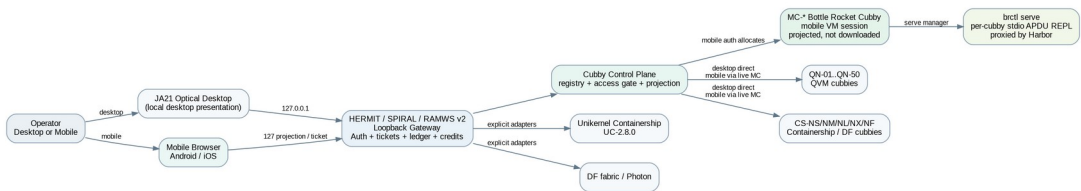


Figure 1 - Updated Harbor Bridge Terminal system context, including Cubby registry, mobile Bottle Rocket sessions, Qnode/containership cubbies, and the brctl serve proxy.

|                   |                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| Report date       | 25 September 2026                                                                                                          |
| Analyzed artifact | Harbor-Bridge-Terminal-main (2).zip                                                                                        |
| Snapshot scope    | Updated uploaded archive plus fresh sandbox execution of supported Node/Python paths, including Cubby/mobile gateway flows |
| Evidence posture  | Documented design + archived evidence + fresh reproduction + derived analysis kept distinct                                |

Purpose: technical explanation and evidence review; not a production certification. External products referenced only by local path pointers are not assumed to be present in the uploaded artifact.

## Executive Summary

Harbor Bridge Terminal is an integrated local workstation that combines a custom terminal stack (HERMIT), an in-memory shell and virtual filesystem (SPIRAL), a bounded WebSocket gateway (RAMWS v2), a Unikernel Containership corpus, a 50-node QVM fleet abstraction, DF fabric bindings, a JA21 browser launcher, and now a unified Cubby control plane for desktop and mobile execution slots. The updated snapshot extends the original local-terminal concept into a browser-projected mobile model: mobile authentication allocates a Bottle Rocket VM cubby (MC-\*), returns its projection URL in the ticket response, and requires that live mobile cubby as the gateway path before a mobile client may attach to QVM or containership cubbies.

The updated snapshot contains the complete Cubby gateway/control-plane source, a Windows brctl materializer/serve binary, mobile compiler and registry logic, archived Windows verification artifacts, and enough Node code to reproduce the Cubby admission/projection path on the review host. It is still not fully self-contained: the full Bottle Rocket, iOS735\_LCTL, LinearAndroid\_LCTL, RODEO, QVM, and JA21 product trees remain external products referenced through PRODUCT\_LINK metadata. Consequently the gateway-level Cubby behavior can be freshly reproduced here, while full mobile-VM image materialization, Windows brctl execution, QVM fleet execution, and JA21 launch behavior must be separated into archived evidence or integration-contract analysis.

### Key evidence result

The archived R28 experiment remains the strongest quantitative performance backtest: 42.5% lower W1 CPU per echoed keystroke, 67.5% lower W2 CPU per MiB of terminal output, and 99.21% lower W3 CPU per paste for the RAMWS v2 + worker-thread candidate versus the v1 + process-worker baseline. The updated mobile path adds an orthogonal validation result: the Cubby gateway workflow reproduced 12/12 checks fresh on Linux, while the archived Windows brctl serve proxy passed 8/8 checks including a real APDU response through the Harbor proxy.

*Table 1 - Executive evidence synopsis*

| Area                       | Most important observation                                                                                                                                     | Evidence class                |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Bridge syntax              | 90/90 JavaScript files passed the updated-snapshot syntax gate.                                                                                                | Fresh reproduction            |
| Archived full bridge suite | 127/127 tests passed in the preserved full-suite TAP artifact.                                                                                                 | Archived evidence             |
| Mobile Cubby gateway       | 12/12 fresh checks passed: MC allocation, projection, desktop direct path, mobile BR prerequisite, QN attach-via-BR, ticket enrichment, and registry listing.  | Fresh reproduction            |
| brctl serve proxy          | 8/8 archived Windows checks passed. Fresh Linux gateway/projection checks pass, but the Windows PE brctl.exe cannot execute on Linux.                          | Archived + fresh host-limited |
| Broad bridge suite         | A current npm test run reached test 54 with no observed failures before the review cutoff; six DF cases skipped because external DF containers were not bound. | Fresh partial reproduction    |

| Area                      | Most important observation                                                                                                                              | Evidence class      |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| R28 performance backtest  | W1/W2/W3 CPU reductions 42.5% / 67.5% / 99.21%; candidate p50 launch means ~0.036 ms; throughput ~27.8 MiB/s.                                           | Archived experiment |
| Qnode fleet               | Archived audit records 50/50 inventory/load pass and 1,750 QVM selftests; 51,200 Bell-simulation shots are statistically consistent with a 50/50 split. | Archived + derived  |
| Containership edge corpus | 9,833 pass, 41 fail, 46 skip; 59 suite-green, 27 suite-red; failures are classified rather than hidden.                                                 | Archived evidence   |
| Release provenance        | Current bridge manifest: 275 matching entries, 14 mismatched/missing, 7 unlisted files; four new unlisted files are Cubby/mobile gateway modules.       | Fresh reproduction  |
| Load harness              | 1- and 4-session runs complete; default 8-session run conflicts with the unchanged 4-connection-per-principal limit.                                    | Fresh reproduction  |

## Overall engineering interpretation

The design shows unusually explicit treatment of bounded memory, cumulative byte credits, deterministic session exit, transport framing limits, and provenance of performance claims. The strongest technical story is the RAMWS v2 gateway: its protocol, allocation ledger, admission controls, and historical experiment are coherent and mutually reinforcing. The main weaknesses visible in the present snapshot are release-sealing drift, several test/benchmark assumptions that lag the thread-backend implementation, and archive portability gaps for externally linked QVM/JA21 products. None of those findings invalidates the architecture; they do constrain what can be called freshly reproducible from this ZIP alone.

## Evidence language used in this note

*Table 2 - Evidence classification*

| Label      | Meaning                                                                                                        |
|------------|----------------------------------------------------------------------------------------------------------------|
| [SOURCE]   | A design or behavior described by repository source code or documentation.                                     |
| [ARCHIVED] | A preserved test, benchmark, audit, screenshot, or JSON/TAP result generated before this review.               |
| [FRESH]    | A command executed during this review against the uploaded working tree.                                       |
| [DERIVED]  | A calculation performed from archived/fresh measurements (for example means, chi-square, or file composition). |

## Contents

1. Scope, terminology, and methodology
2. Platform composition and archive provenance
3. System architecture and boot orchestration
4. HERMIT terminal and SPIRAL execution model
5. RAMWS v2 wire protocol and session semantics
6. Resource model, memory ledger, and backpressure
7. Security architecture and trust boundaries
8. DF, Photon, JA21, and Qnode integration
9. Mobile Cubby architecture and integration
10. R28 performance backtest and ablation
11. Qnode fleet audit and Bell-distribution analysis
12. Fresh verification campaign
13. Load benchmark and current harness defects
14. Release integrity and provenance drift
15. Containership / edge evidence
16. Reliability, recovery, and operational characteristics
17. Engineering findings and remediation priorities
18. Reproduction guide
- Appendix A. Protocol reference
- Appendix B. Evidence inventory and limitations

# 1. Scope, Terminology, and Methodology

## 1.1 Scope

This note explains the Harbor Bridge Terminal platform at system, protocol, resource, security, integration, and verification levels. The objective is expository: to make the design legible to an engineer who did not build it, while also testing enough of the current snapshot to distinguish implementation evidence from documentation claims.

The review treats the uploaded ZIP as the primary artifact. Static inspection covers repository documentation, source, manifests, experiment configuration/results, fleet audits, TAP logs, and screenshots. Fresh execution covers the bridge syntax gate, selected Node test groups, repeated isolated gateway tests, RAM quota behavior, load-benchmark cases that are internally consistent, release-manifest verification, and read-only containership status commands.

## 1.2 Meaning of "backtesting" in this report

The platform does not expose a financial trading strategy or historical market-data engine in the examined snapshot. Accordingly, no financial backtest is invented. Here, "backtesting" means revision-to-revision and baseline-to-candidate regression measurement: replaying controlled terminal workloads against a frozen experiment design, comparing CPU, latency, throughput, fidelity, and backend variants. The archived R28 experiment is the principal backtest because it explicitly defines baseline and candidate arms and preserves result JSON.

## 1.3 Review methodology

1. Inventory the ZIP from its central directory so pristine file counts and sizes are not contaminated by generated runtime artifacts.
2. Trace boot entrypoints from START\_HARBOR.cmd through start-local.js, principal-token creation, gateway startup, product discovery, and JA21 handoff.
3. Trace data flow from renderer input through WebSocket framing, session ownership, worker/SPIRAL execution, output gating, credits, and renderer consumption acknowledgements.
4. Read the security and resource-model documents alongside implementation tests; treat the worker thread as an execution isolation boundary, not an OS security boundary.
5. Parse archived R28 experiment results, Qnode fleet audit JSON, edge test evidence, release manifest, and full-suite TAP artifacts.
6. Execute safe current-tree gates and isolate flaky/timing-sensitive tests rather than interpreting a concurrent stress failure as a deterministic feature failure.
7. Perform derived statistical calculations only where the underlying archived counts are available and clearly scoped.

### Important reproducibility boundary

The current extracted containership tree was allowed to create temporary runtime material during a terminated verify attempt. Archive-composition figures therefore come from the pristine ZIP central directory, not post-execution disk usage. Full current containership verification is not claimed.

## 1.4 Test environments must not be conflated

Table 3 - Evidence environments

| Evidence set            | Host / runtime                                                                               | Purpose                                                   |
|-------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| R28 archived experiment | Linux 6.18.44, 2 vCPU, Intel Xeon 2.80 GHz, Node v22.22.2                                    | Controlled baseline/candidate performance comparison.     |
| Fresh bridge review     | Linux 6.18.44, 5 visible CPUs, Intel Xeon Platinum 8370C 2.80 GHz, Node v22.16.0, npm 10.9.2 | Functional verification and local loopback diagnostics.   |
| Archived Qnode audit    | Original Windows Harbor development tree; Python QVM CLI per node                            | Fleet inventory, independence, selftest, Bell simulation. |
| Archived edge evidence  | Linux, Python 3.11.15, pytest, 6 jobs                                                        | Containership edge-atom regression evidence.              |

## 2. Platform Composition and Archive Provenance

At the top level, Harbor is an integration workspace. The root README describes the containership as the harbor yard and the terminal as the bridge: the operator can inspect the containership, open bounded terminal sessions, focus into Qnodes, and drive DF fabric operations from one console. The primary user-facing local URL is intentionally opened in JA21 rather than the system browser.

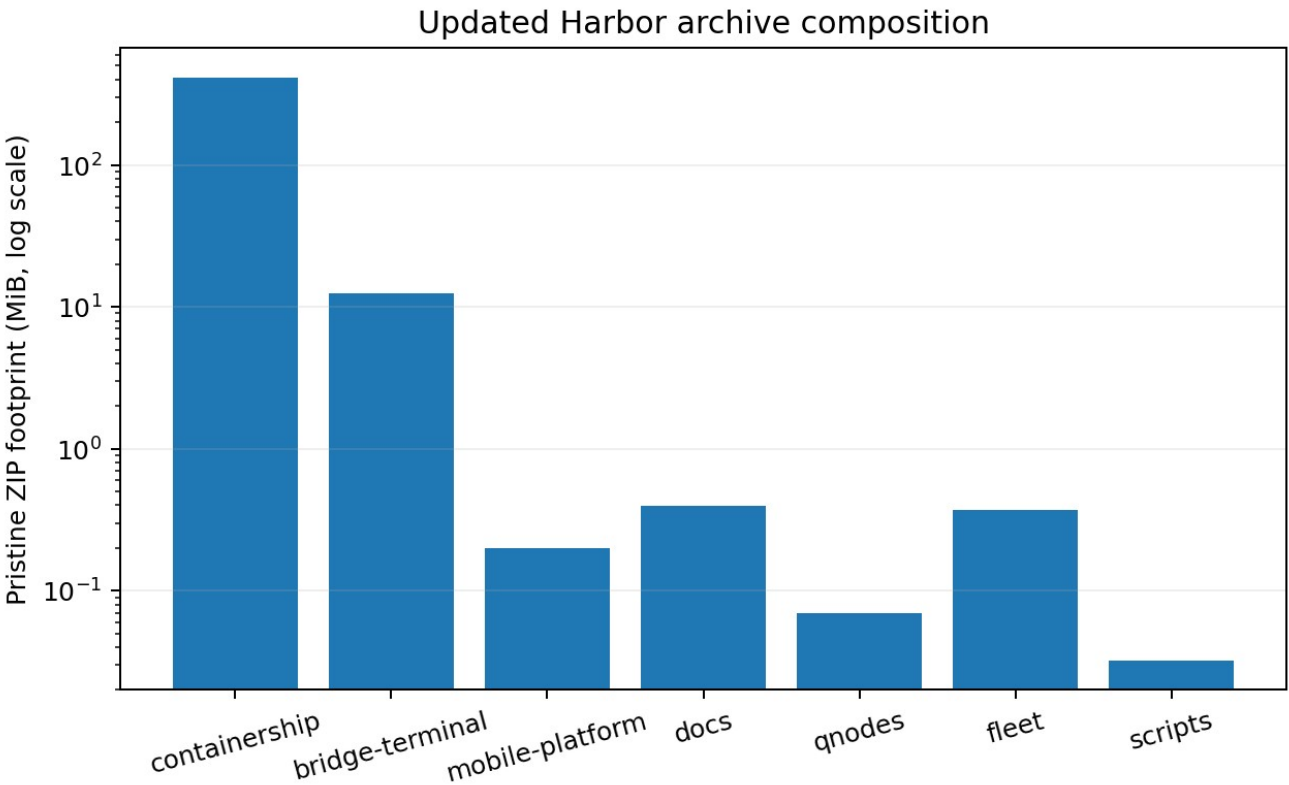


Figure 2 - Pristine updated ZIP composition by uncompressed footprint. The containership dominates; bridge-terminal remains the principal executable control-plane source, while mobile integration adds a compact control layer and verification corpus.

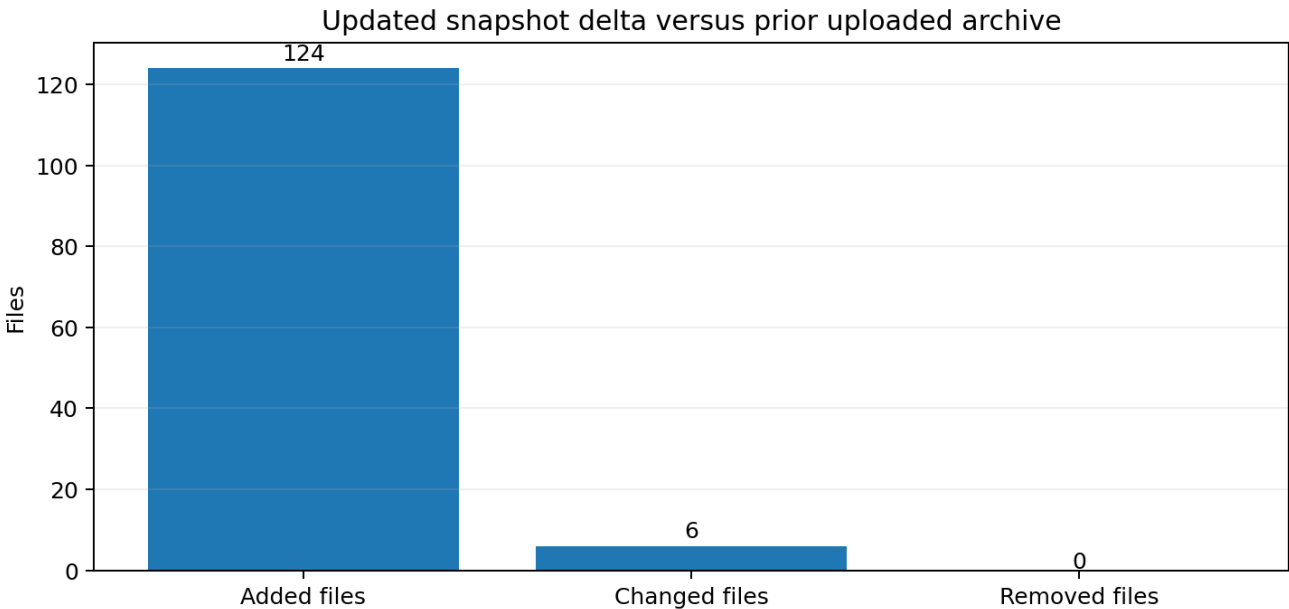


Figure 3 - File-level delta from the prior uploaded snapshot: 124 files added, 6 changed, and none removed. The additions are concentrated in mobile/Cubby integration and verification.

| Top-level area | Files / role                                    | Pristine ZIP footprint |
|----------------|-------------------------------------------------|------------------------|
| containership  | 20,165 files; UC-2.8.0 corpus and edge evidence | 414.55 MiB             |

| Top-level area  | Files / role                                                               | Pristine ZIP footprint  |
|-----------------|----------------------------------------------------------------------------|-------------------------|
| bridge-terminal | 291 files; HERMIT/SPIRAL/RAMWS plus Cubby gateway code                     | 12.46 MiB               |
| mobile-platform | 31 files; mobile-node metadata, compiler, registry support, brctl.exe      | 0.198 MiB               |
| docs            | 82 files; technical note, mobile verification, SPIRAL evidence             | 0.396 MiB               |
| fleet           | 5 files; Harbor roster, mobile platform, Cubby registry, Qnode audit       | 0.369 MiB               |
| qnodes          | 201 files; Harbor metadata stubs for 50 nodes                              | 0.069 MiB               |
| scripts         | 12 files; desktop/mobile linking, materialization, delivery, audit helpers | 0.032 MiB               |
| Entire ZIP      | 20,796 files                                                               | 428.10 MiB uncompressed |

## 2.1 Integrated component inventory

*Table 5 - Platform component matrix*

| Component              | Role                                                                                                                        | Snapshot status                                                                              |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| HERMIT                 | Desktop/browser virtual terminal, ANSI/VT rendering, UI/controller.                                                         | Source present.                                                                              |
| SPIRAL                 | In-memory shell backend, parser, VFS, command registry, line editor.                                                        | Source present.                                                                              |
| RAMWS v2               | Loopback WebSocket gateway, binary data framing, credits, session/ledger controls.                                          | Source present; freshly testable.                                                            |
| UC-2.8.0 Containership | Edge-atom corpus, CLI workflows, platform/master-workflow evidence.                                                         | Corpus present; status commands freshly runnable.                                            |
| Qnode fleet            | 50 independent QVM copies addressed as qn01..qn50.                                                                          | Full copies absent from uploaded ZIP; archived fleet audit present.                          |
| DF fabric              | DF_Fabric plus N_SMALL/N_MEDIUM/N_LARGE/N_XLARGE bindings.                                                                  | Integration code present; external packages not bound here.                                  |
| VEC1 Photon            | Optional loopback compute/control plane used for delegated fabric workflows.                                                | Client/workflow integration source present.                                                  |
| VB-JA21 9.8.7          | Standalone optical desktop / omni-bin browser for Harbor local URL.                                                         | Only link metadata present; portable product absent.                                         |
| Mobile Cubby platform  | Bottle Rocket MC-* session allocation, local-127 browser projection, QN/CS mobile access gate, per-cubby brctl serve proxy. | Gateway/compiler source and Windows brctl.exe present; external mobile product trees absent. |

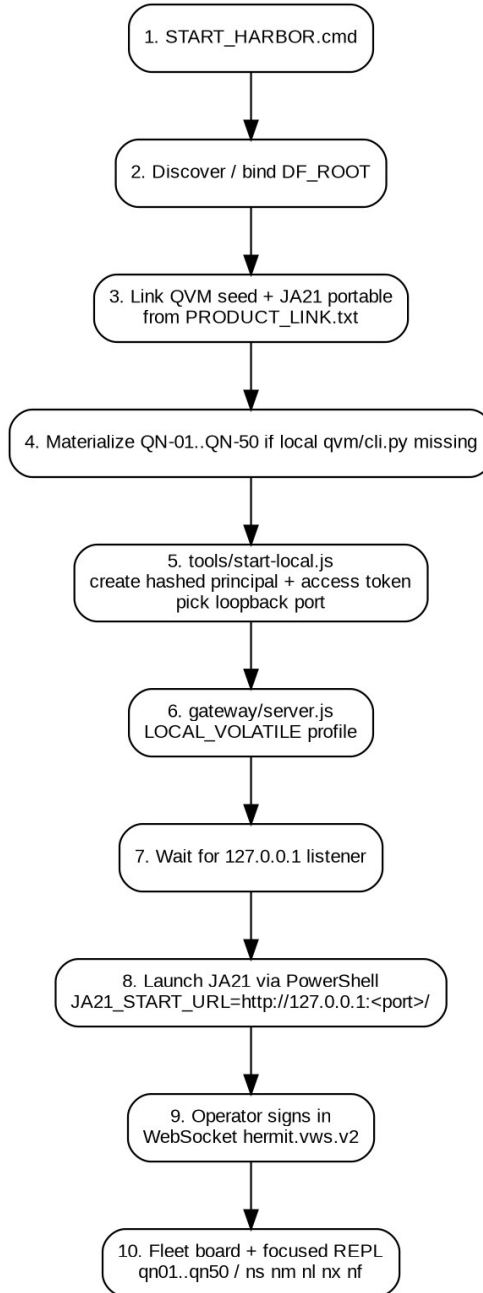
## 2.2 Portability gap in the uploaded snapshot

The root documentation describes each Qnode as a complete, independent QVM 8.1.0-alpha tree. In the uploaded ZIP, each QN-XX directory contains only Harbor metadata and launch scaffolding; the expected qvm/cli.py product tree is not included. The qvm/PRODUCT\_LINK.txt file points to an absolute path on the original Windows machine, and the JA21 PRODUCT\_LINK similarly points to a Downloads tree on that machine. The updated mobile platform follows the same pattern: Bottle Rocket, iOS735\_LCTL, LinearAndroid\_LCTL, and RODEO are represented by link metadata rather than committed product trees. Those links are useful integration evidence but are not portable dependencies.

### Implication

The archived 50/50 Qnode load audit can be analyzed as evidence of an earlier full-materialized state, but it cannot be rerun faithfully from this ZIP in the present environment without supplying the missing QVM product. The same distinction applies to JA21 and to full Bottle Rocket/iOS/Android/RODEO materialization. The Cubby gateway contract itself is portable and was reproduced independently.

### 3. System Architecture and Boot Orchestration



*Figure 4 - Harbor boot sequence from Windows launcher through external-product binding, Qnode materialization check, gateway startup, and JA21 navigation.*

#### 3.1 Windows entrypoint

`START_HARBOR.cmd` is an orchestration launcher rather than the terminal itself. Its job is to construct the local integration environment: discover a DF root when available, restore QVM, optical-desktop, and mobile-platform product links, ensure Qnode materialization, then hand control to `bridge-terminal/tools/start-local.js`. Flags allow browser suppression, fabric suppression, port preference, and reuse of an already listening local gateway.

## 3.2 Local bootstrap and principal provisioning

The Node bootstrap creates `bridge-terminal/.vws-local/principals.json` if it does not already exist. The plaintext operator token is written to a gitignored `ACCESS_TOKEN.txt` while only a SHA-256 token hash is kept in the principal database. This design makes first-run local sign-in convenient without checking an operator secret into the repository. Losing the token requires deleting the local principal/token state and regenerating it.

The gateway binds on loopback, preferring ports 10000 through 10019. The bootstrap discovers DF and Qnode roots, starts the gateway, waits for listening readiness, then launches JA21 with `JA21_START_URL` and `JA21_ALLOW_PRIVATE_HOSTS=1`. It does not call a generic shell URL opener, so the intended browser surface remains the designated Harbor optical desktop.

## 3.3 Fleet focus model

The terminal exposes short focus codes that turn the general console into a target-oriented control plane: `qn01` through `qn50` for Qnodes and `ns/nm/nl/nx/nf` for the four DF node classes and fabric. The focus mechanism is an interaction convention layered on explicit target discovery and runner code; it does not collapse those products into one process.

# 4. HERMIT Terminal and SPIRAL Execution Model

## 4.1 HERMIT VT architecture

HERMIT implements a conventional terminal decomposition: a parser transforms byte/character streams into terminal actions, a screen model stores terminal state, and a canvas renderer paints the model. The parser is described as DEC VT500-like and handles C0 controls, ESC/CSI/OSC sequences, SGR attributes, 256-color and truecolor forms. The screen model tracks cursor state, pen attributes, a scrolling region, tab stops, deferred autowrap, and a bounded 5,000-line scrollback ring.

This matters because the SPIRAL backend can emit the same style of ANSI redraw sequences that a PTY-oriented terminal would consume. Interactive line editing therefore does not require a browser-specific edit model: arrow movement, history, Home/End, Ctrl-U/Ctrl-K/Ctrl-W edits, Ctrl-L clear, and tab completion are expressed through the terminal contract.

## 4.2 SPIRAL kernel loop

1. Construct and emit the shell prompt (including PS1 substitutions and SGR).
2. Read one interactive line using the LineReader, with editing/history/completion.
3. Parse quoting, expansion, sequencing, pipelines, redirections, and conditionals.
4. Resolve commands through the registry and execute against the in-memory VFS and allowed host adapters.
5. Stream stdout/stderr as ANSI, record the exit status, and return to the prompt.

## 4.3 Virtual filesystem and command surface

SPIRAL uses an in-memory VFS with absolute/relative path resolution, `.` and `..` normalization, `stat/list/mkdir/move/copy/remove` operations, and a seeded Unix-like directory tree. The ordinary shell filesystem does not directly expose the host filesystem. Host interaction is instead concentrated in

explicit commands/adapters for DF, Photon, browser navigation, and Qnodes. This separation is a major part of the trust model: a general shell operation is not automatically a host capability.

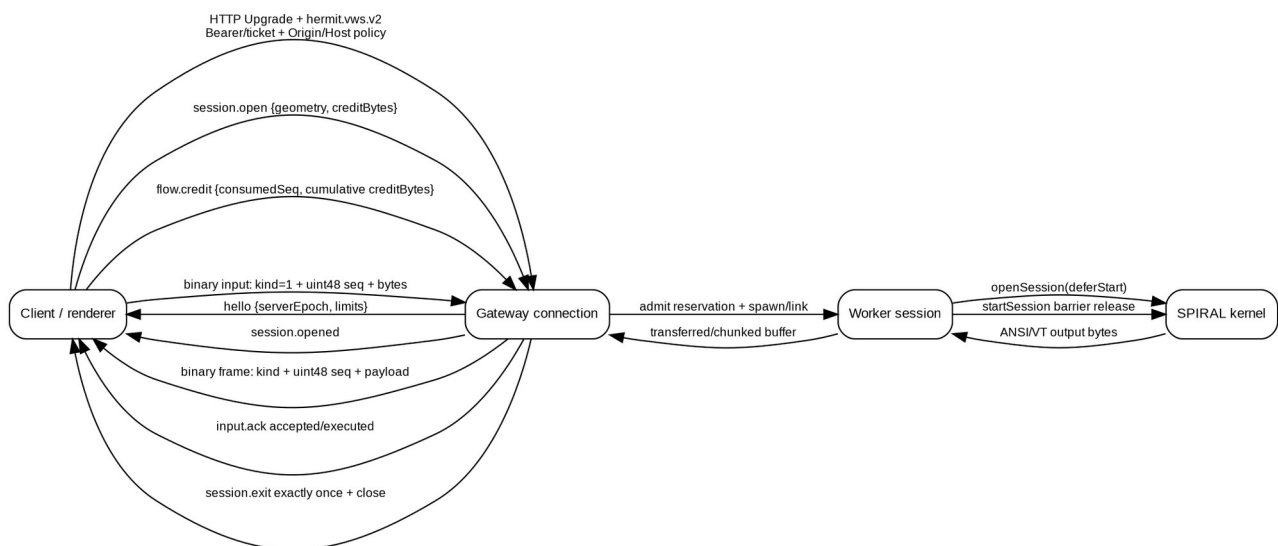
*Table 6 - SPIRAL command-surface categories*

| Command family | Representative commands                                     | Engineering significance                                     |
|----------------|-------------------------------------------------------------|--------------------------------------------------------------|
| Shell          | help, man, env, export, alias, history, date, sysinfo, exit | REPL environment and deterministic session control.          |
| Filesystem     | pwd, cd, ls, tree, find, mkdir, rm, cp, mv, stat, write     | Exercises the in-memory VFS rather than arbitrary host disk. |
| Text/coreutils | echo, cat, grep, sort, wc, head, tail, seq, sleep           | Pipeline and streaming workload surface.                     |
| Browser        | browser open/go/close/info, open                            | Narrow browser adapter seam.                                 |
| Fabric/DF      | df, fabric, node                                            | Explicit external CLI / Photon delegation.                   |
| Qnode          | qn status/where/attach; qnXX focus codes                    | Explicit QVM fleet routing.                                  |

## 4.4 Bounded command behavior

Several command implementations impose bounds that are relevant to resource safety. For example, the seq command refuses a request for 4,000,000 items with a limit message rather than attempting unbounded capture. This exact behavior becomes important later: one fresh RAM quota test expected the words "capture limit" or "quota" and timed out even though the command was in fact bounded by a more generic "exceeds the limit" diagnostic.

## 5. RAMWS v2 Wire Protocol and Session Semantics



*Figure 5 - RAMWS v2 control/data flow: authenticated connection, session opening, binary input/output sequence numbers, cumulative acknowledgements, and client credits.*

## 5.1 Two-plane framing

RAMWS v2 deliberately separates a small JSON control plane from binary terminal data. Control frames are text JSON objects with a version, type, payload, optional request id, and session/epoch coordinates where required. The implementation documents a 65,536-byte control-message cap plus strict UTF-8, duplicate-key, and nesting/depth handling. Terminal input and output travel as binary frames, avoiding base64 overhead in the current protocol.

*Table 7 - RAMWS v2 frame model*

| Plane / kind         | Encoding        | Key fields / limits                                                                                   |
|----------------------|-----------------|-------------------------------------------------------------------------------------------------------|
| Control              | UTF-8 JSON text | {v:2,type,payload,rid?,sid?/epoch?}; <= 65,536 bytes; strict validation.                              |
| Input data (kind=1)  | Binary          | byte 0 kind; bytes 1..6 unsigned 48-bit big-endian sequence; remaining bytes payload; <= 8,192 bytes. |
| Output data (kind=2) | Binary          | same header layout; output payload <= 16,384 bytes.                                                   |
| Zero-length data     | Rejected        | Avoids meaningless sequence advancement and ambiguous accounting.                                     |

## 5.2 Session binding and epoch semantics

Binary data frames do not carry a session identifier. They are interpreted in the authenticated session context already bound to that connection. A hello message communicates the server epoch, profile, and limits. A session.open request establishes initial output credit and may carry prior-session metadata for reset semantics, but the design does not promise replay. When a connection is re-established, the protocol can explicitly send session.reset before opening a fresh session, preserving the principle that unacknowledged input is not silently replayed.

## 5.3 Three acknowledgement levels

The protocol differentiates acceptance, execution, and consumption because they answer different reliability questions:

*Table 8 - Cumulative acknowledgement semantics*

| Acknowledgement | Meaning                                                    | Why it matters                                                             |
|-----------------|------------------------------------------------------------|----------------------------------------------------------------------------|
| acceptedSeq     | Input bytes have been copied into bounded server RAM.      | Allows coalescing while proving ownership/accounting transfer.             |
| executedSeq     | Input has been delivered/completed through the shell path. | Provides a stronger application-progress boundary.                         |
| consumedSeq     | Client reports output handed to the renderer/consumer.     | Closes the end-to-end flow-control loop and controls credit replenishment. |

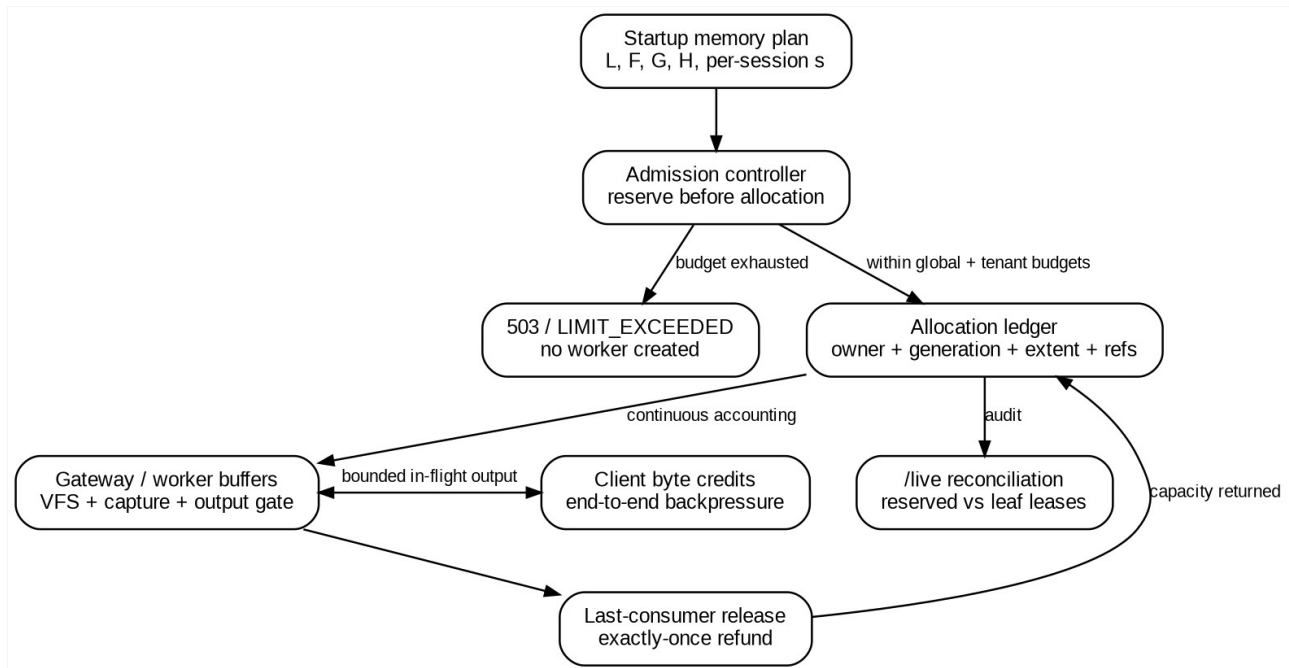
## 5.4 Output ordering and terminal exit

Output sequence numbers are required to be contiguous from one, and a session emits exactly one session.exit. That combination simplifies client logic: there is one monotone output stream and one terminal state transition. The gateway also limits outstanding output messages and enforces a maximum pause duration under prolonged credit starvation; a client that never consumes cannot grow server output indefinitely.

## 5.5 RFC6455 parser hardening

The WebSocket parser is designed around early bounding. It applies size checks before expensive assembly, unmask one-chunk frames in place, allocates one owned buffer for fragmented messages, uses a 4/16/64 KiB slab pool, applies an assembly deadline, and rate-limits control frames. These are defensive implementation choices aimed at preventing a nominally local endpoint from becoming an allocation or parser-amplification sink.

# 6. Resource Model, Memory Ledger, and Backpressure



*Figure 6 - Memory admission, ledger accounting, buffer ownership, client credits, reconciliation, and exactly-once capacity return.*

## 6.1 Startup memory plan

RAMWS treats memory capacity as a pre-admission resource rather than something to discover after allocation. The documented symbols are: L, the smallest finite memory boundary; F, gateway startup RSS; G, idle pool allowance; H, reserved headroom; and s, the per-session reservation. The computed session ceiling is:

**Admission equation**

```

N_ceiling = floor((L - F - G - H) / s)
N_admitted = min(N_configured, N_ceiling)

```

Transport memory is reserved before the HTTP 101 upgrade and worker memory is reserved before worker creation. If budget is exhausted, the system can return an HTTP 503 or a protocol LIMIT\_EXCEEDED response without first creating the resource that would violate the budget. That ordering is one of the most important aspects of the model.

**6.2 Allocation ledger**

The ledger models owned memory using explicit descriptors that include operator, buffers, formats, budget, ownership, completion, quality, and backend metadata. Accounting checks session, tenant, and global levels atomically. Capacity, rather than current string/array length, is the relevant charge. Shared views increment references rather than charging full capacity again, and the final consumer releases the underlying extent exactly once.

**6.3 Lifecycle**

The documented lifecycle is CREATED -> VALIDATED -> RESERVED -> ACTIVE -> DRAINING -> COMPLETED -> RELEASED, with REJECTED and FAILED terminal branches. This is stronger than a simple allocation counter because it gives the implementation a place to enforce validation-before-reservation, reserve-before-activation, and release-after-last-consumer invariants.

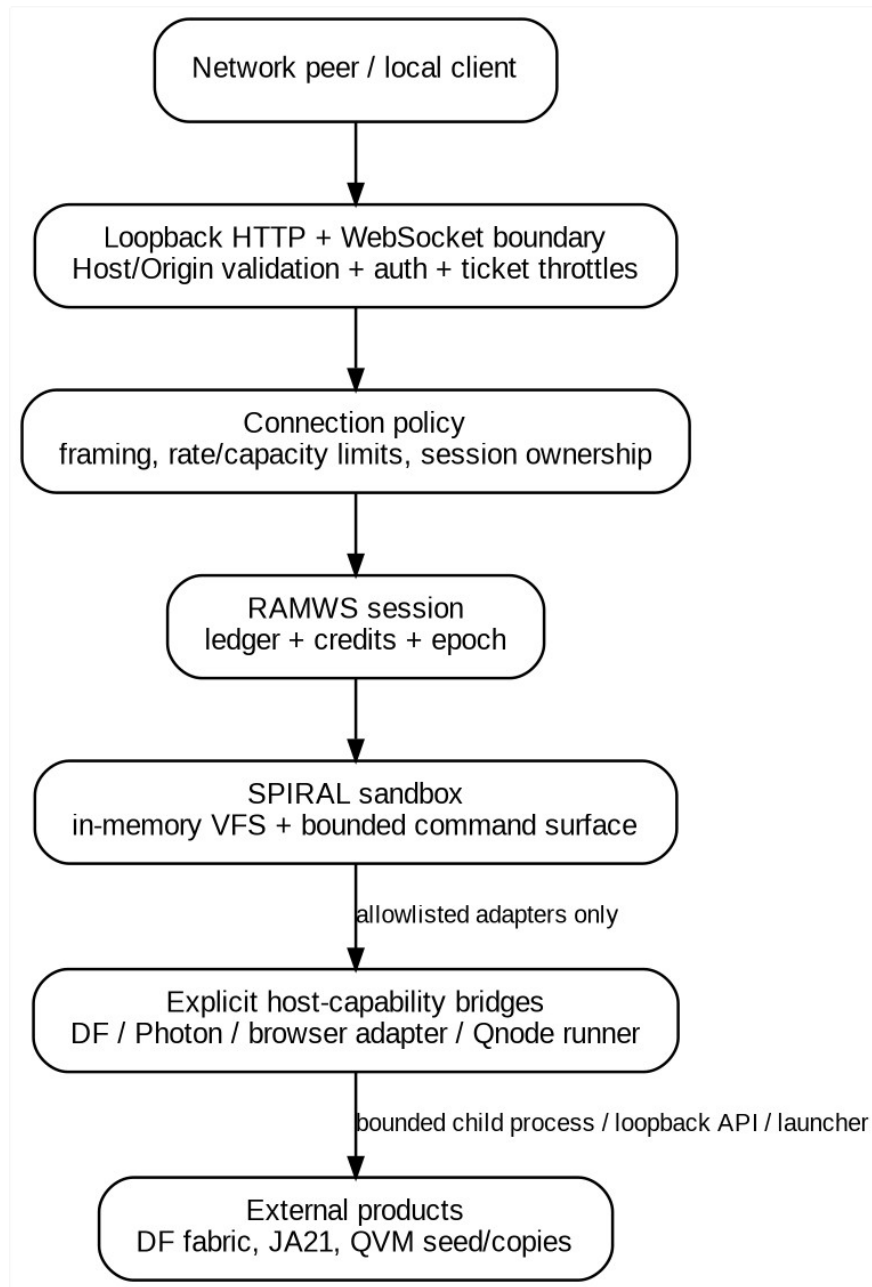
**6.4 End-to-end byte credits**

Output credits are cumulative, monotone, and bounded by one configured window. Exhausted credit pauses the output path end-to-end. The documentation additionally caps outstanding output messages at 1,024 and allows a maximum pause of roughly 120 seconds before the session is terminated for quota behavior. This transforms a slow or malicious consumer from an unbounded buffering problem into a bounded liveness decision.

**6.5 Worker-heap enforcement caveat**

Node worker resourceLimits are not absolute if a process-wide NODE\_OPTIONS --max-old-space-size takes precedence. The gateway therefore probes whether the requested heap cap is actually enforceable and may refuse startup if the configured safety promise cannot be upheld. In the fresh environment the probe reported heapCap=ENFORCED.

## 7. Security Architecture and Trust Boundaries



*Figure 7 - Security/trust boundaries from loopback peer through connection policy and SPIRAL sandbox to explicit external-product adapters.*

### 7.1 Boundary model

Harbor is designed for local operation, but it does not equate loopback with trust. The HTTP/WebSocket boundary validates Host and Origin, authenticates principals/tickets, applies per-principal connection/session caps, throttles ticket attempts, limits ping/control activity, enforces capabilities, and supports revocation. The renderer is configured with contextIsolation, disabled Node integration, and Electron sandboxing. Main-process IPC checks the trusted top-level frame, exact payload shapes/ranges, and session ownership.

## 7.2 The worker thread is not an OS sandbox

The security documentation is explicit that a Node worker thread is not an operating-system security boundary. The platform instead relies on several narrower boundaries in combination: SPIRAL and its in-memory VFS, environment allowlisting, command filtering, fabric policy, protocol quotas, and the limited host-capability adapter surface. This is the correct way to describe the model; calling the worker itself a security sandbox would overstate the isolation.

## 7.3 Threat/control mapping

*Table 9 - Threat-to-control map*

| Threat / failure mode              | Primary controls                                                                    | Residual note                                                             |
|------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Unauthorized local client          | Token/ticket auth, Host/Origin checks, principal ownership, revocation              | Loopback still shares a host trust domain; local secret handling matters. |
| Oversized/malformed frames         | Strict JSON/UTF-8 rules, binary frame caps, fragmentation deadline, early rejection | Parser correctness remains security-critical.                             |
| Slow consumer / memory growth      | Byte credits, bounded queues, max pause, admission/ledger budgets                   | Trades unbounded memory for controlled session termination.               |
| Renderer compromise                | contextIsolation, nodeIntegration=false, sandbox, IPC frame/payload checks          | Browser/Electron/WPF integration still inherits runtime vulnerabilities.  |
| General shell reaches host disk    | In-memory VFS; explicit host adapters rather than host shell                        | Adapter commands become the privileged review surface.                    |
| External DF/QVM/JA21 product drift | Pinned links/manifests/audits where available                                       | Current uploaded ZIP lacks the full external products.                    |

## 7.4 Non-durability is a deliberate property

RAMWS is documented as LOCAL\_VOLATILE: it does not promise durable session replay. This removes an entire class of disk-spool and recovery semantics but requires clients to understand reset behavior. The reconnect screenshot later in this report shows that unacknowledged inputs are intentionally not replayed into a new session.

# 8. DF, Photon, JA21, and Qnode Integration

## 8.1 DF fabric

HERMIT is a control surface for the offline DF fabric rather than the fabric implementation itself. Discovery resolves DF\_Fabric and the N\_SMALL/N\_MEDIUM/N\_LARGE/N\_XLARGE nodes. Read-only terminal commands can show topology/status/specs; runner commands can invoke build/run/verify workflows through the DF CLIs. Network policy in the DF descriptors is described as deny/offline, and HERMIT does not add arbitrary fabric network access.

## 8.2 VEC1 Photon delegation

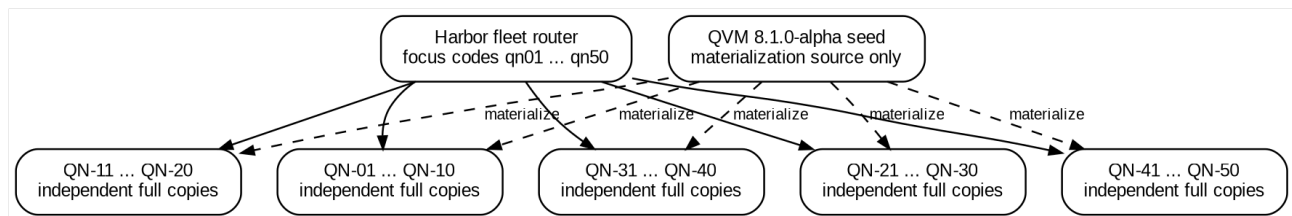
When a VEC1 Photon is bound, selected fabric operations can be retargeted to a loopback JSON-RPC control plane instead of executing heavy work in the Electron main process. The intended workflow

performs electron lifecycle work first, then obtains the fabric diagnostic/proof. Mutating requests carry an X-VEC1-Token. The code preserves the fabric witness as raw data rather than coercing a 64-bit value through an unsafe JavaScript number conversion.

### 8.3 JA21 optical desktop

Harbor treats JA21 as the preferred presentation shell. The launcher passes the local Harbor URL through JA21\_START\_URL / -StartUrl and enables the explicit private-host exception required for 127.0.0.1. The repository also retains an Electron BrowserView adapter seam as a secondary option. In the supplied ZIP, the JA21 portable itself is absent and is represented by a PRODUCT\_LINK path, so the launch contract can be inspected but not freshly exercised end-to-end.

### 8.4 Qnode fleet

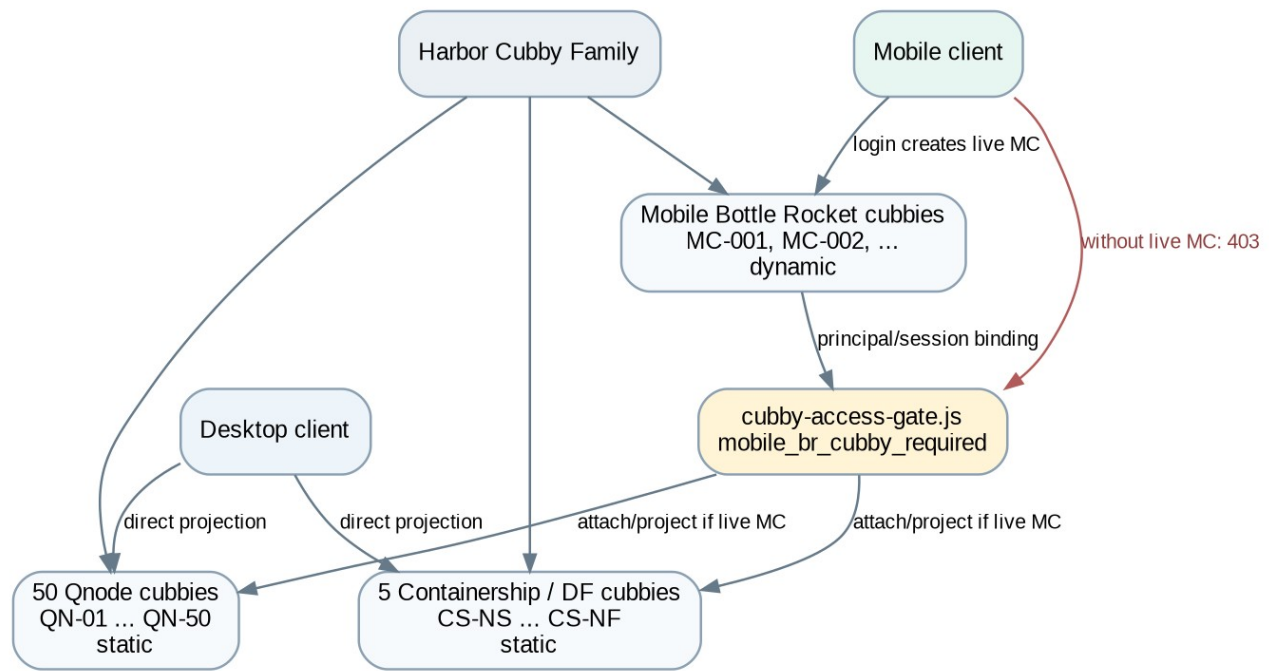


*Figure 8 - Intended 50-node fleet model: qn01..qn50 route to independent QVM copies; the QVM seed is used only to materialize copies, not as their runtime dependency.*

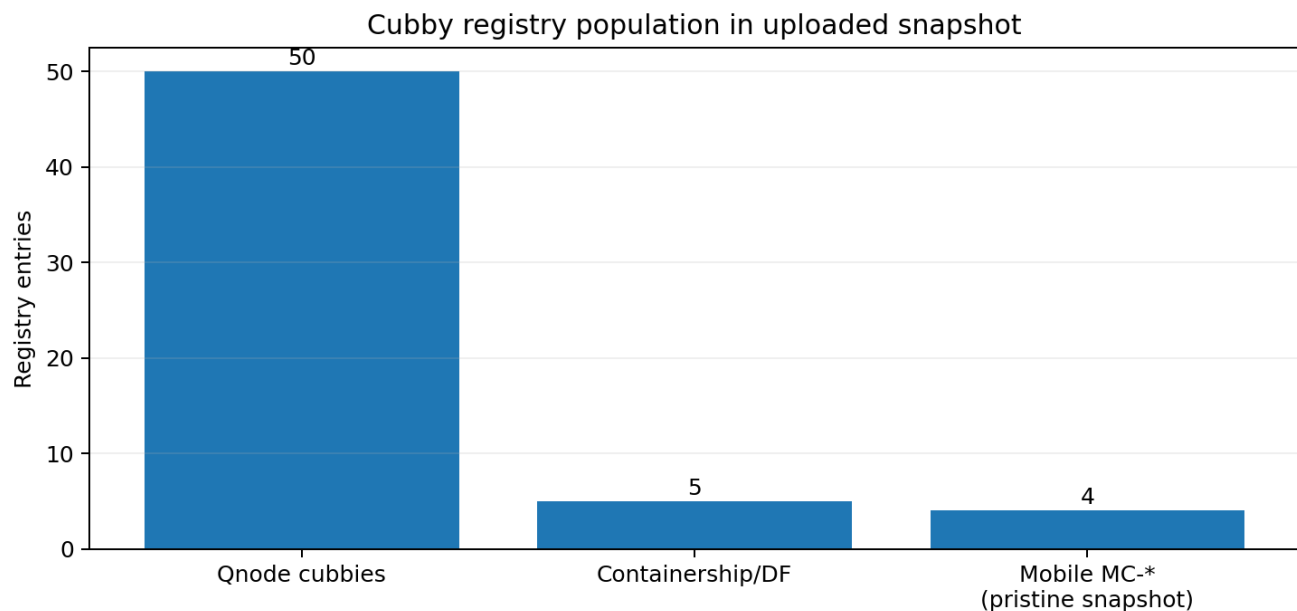
The design goal is genuine file-level independence among nodes: materialization copies the seed into fifty QN-XX trees and QNODE.cmd runs the CLI with that copy as the working/PYTHONPATH root. The archived fleet audit includes a simple independence marker check in which a marker under QN-01/runtime must not appear under QN-02/runtime. That is not equivalent to OS process isolation, but it is a useful proof against accidental shared-directory aliasing.

## 9. Mobile Cubby Architecture and Integration

The updated snapshot introduces a coherent Cubby abstraction across three execution families: fifty QVM Qnode cubbies (QN-01..QN-50), five containership/DF cubbies (CS-NS, CS-NM, CS-NL, CS-NX, CS-NF), and dynamically allocated mobile Bottle Rocket cubbies (MC-001, MC-002, ...). The central semantic change is that mobile is no longer modeled as a package-download workflow. The primary mobile path is authenticated gateway-side VM-session allocation plus browser projection over Harbor local 127; device download is explicitly false in both the platform manifest and ticket response.



**Figure 9 - Unified Cubby family and access model.** Desktop may project QN/CS directly; mobile must first establish a live Bottle Rocket MC-\* cubby before QN/CS access is admitted.



**Figure 10 - Pristine uploaded registry population:** 50 Qnode cubbies, five containership/DF cubbies, and four historical mobile cubbies. New MC-\* entries append dynamically as mobile logins occur.

## 9.1 Cubby vocabulary and registry invariants

**Table 10 - Unified Cubby classes**

| Cubby class          | Identifiers                       | Lifecycle / access model                                                                                                 |
|----------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| QVM Qnode            | QN-01 .. QN-50                    | Static registry floor; desktop direct projection; mobile requires a live MC-* principal binding before attach/project.   |
| Containership / DF   | CS-NS, CS-NM, CS-NL, CS-NX, CS-NF | Static registry floor; same desktop-direct/mobile-via-BR gate model.                                                     |
| Mobile Bottle Rocket | MC-001, MC-002, ...               | Allocated dynamically on mobile auth; session + projection metadata persisted; operable when projection session is live. |

The pristine fleet/CUBBIES.json declares a static floor of 55 cubbies (50 QN + 5 CS) and a monotone MC-NNN namespace for mobile sessions. Four MC entries are preserved in the uploaded snapshot, making 59 registry entries at archive time. Fresh verification allocated additional review-only MC entries in the mutable extraction; those are test artifacts and are not used as the pristine archive count.

## 9.2 Mobile login, ticket enrichment, and projection



*Figure 11 - Mobile authentication sequence from ws-ticket request through MC allocation, registry/session persistence, ticket enrichment, browser projection, and optional QN/CS attachment.*

A successful POST /api/ws-ticket with X-Harbor-Mobile-Platform set to android or ios crosses the normal authentication boundary first. The mobile-auth-hook then allocates a new MC-\* record, writes cubby-session metadata, and enriches the ticket body with cubby\_id, projection\_url, device\_download=false, and primary=cubby-projection. The browser then opens /cubby/{MC-id}/projection on the same local gateway. The VM/session remains gateway-side; the phone browser receives a projected control surface rather than a VM image download.

The hook is enabled by default and can be disabled with HARBOR\_MOBILE\_AUTH\_HOOK=0. Its default queue mode records work for the cubby materializer without making image compilation a prerequisite for ticket success. Compile/materialize and optional delivery modes are explicit operator choices rather than hidden login side effects.

## 9.3 Desktop-direct versus mobile-via-BR access gate

cubby-access-gate.js distinguishes desktop and mobile clients. Desktop requests to QN-\* or CS-\* use the desktop-direct path. A mobile client targeting QN-\* or CS-\* must have a live MC-\* owned by the same principal; otherwise the gateway returns HTTP 403 with mobile\_br\_cubby\_required. When the principal has a live mobile cubby, the gate returns a mobile-via-br-cubby decision including the BR cubby id, its session id, and projection URL. This creates a concrete admission invariant rather than relying on UI convention.

### Security significance

The mobile prerequisite is enforced at the gateway, not merely suggested by the page. Fresh verification exercised both the 403 negative path and the successful attach-through-live-MC path.

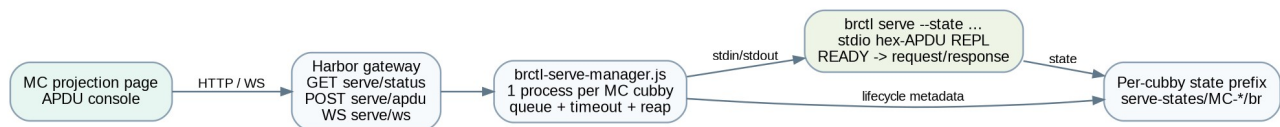
## 9.4 Cubby materializer and external mobile products

fleet/MOBILE\_PLATFORM.json identifies the shared VM substrate as BOTTLE\_ROCKET\_3.0.0\_MODEL\_OPERATIONAL\_110K. The iOS application node is iOS735\_LCTL; the Android application node is LinearAndroid\_LCTL; RODEO is explicitly modeled as a Linear Android sidecar and Gradle substitute, not as a third peer app node. The compiler role is described as a cubby materializer: it resolves linked products, hashes pinned inputs, creates a content-addressed package, optionally runs RODEO diagnostics, invokes brctl assemble when available, and writes COMPILE\_MANIFEST.json. Optional adb/idevice delivery is demoted to sideload and is not the primary enter-Harbor path.

*Table 11 - Mobile/Cubby component roles*

| Mobile component         | Role                                       | Snapshot portability                                               |
|--------------------------|--------------------------------------------|--------------------------------------------------------------------|
| Bottle Rocket 3.0.0 110K | Shared VM substrate / cubby image source   | PRODUCT_LINK only; product tree absent.                            |
| iOS735_LCTL 0.1.0        | iOS Bottle Rocket VM application node      | Metadata/link present; product tree absent.                        |
| LinearAndroid_LCTL 0.1.0 | Android Bottle Rocket VM application node  | Metadata/link present; product tree absent.                        |
| RODEO 0.1.0              | Linear Android Gradle-substitute sidecar   | Metadata/link present; product tree absent.                        |
| brctl.exe                | Windows cubby materializer + serve command | PE32+ x86-64 binary included; SHA-256 b996aaba5fd2a406...e8d1e308. |

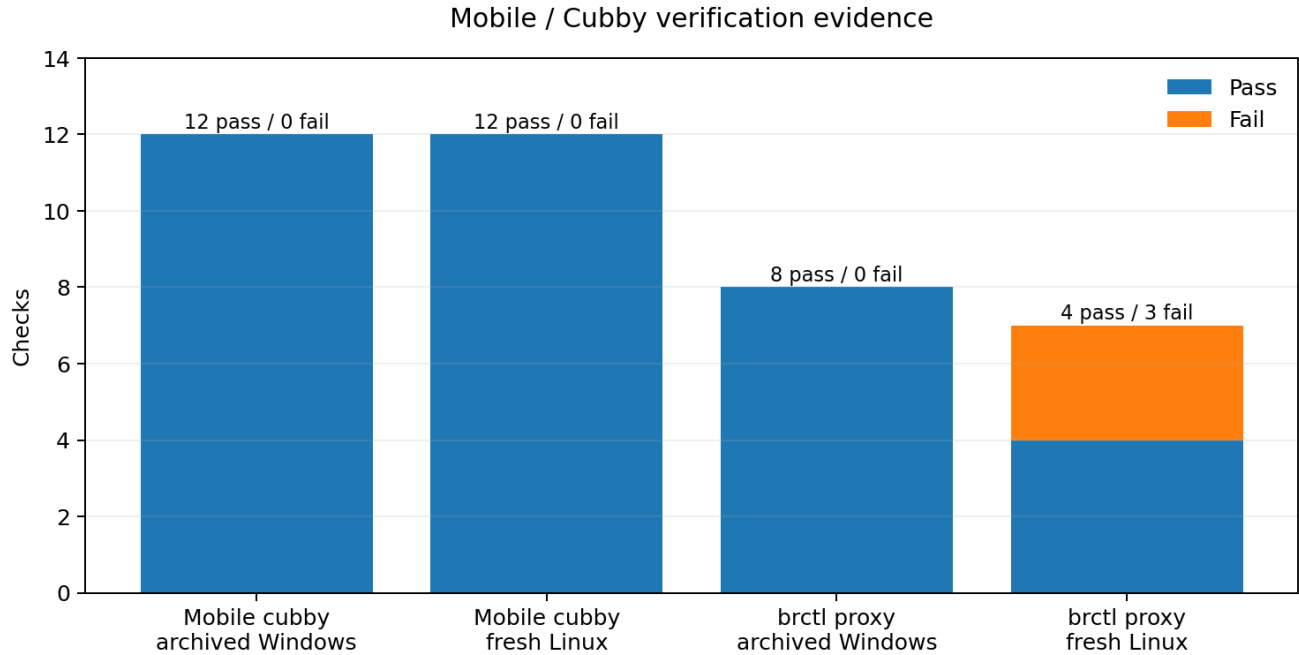
## 9.5 Per-cubby brctl serve proxy



*Figure 12 - Per-mobile-cubby brctl serve architecture. Harbor owns process lifecycle and proxies the stdio APDU REPL through authenticated HTTP/WS endpoints; brctl itself does not open a network listener.*

brctl-serve-manager.js treats brctl serve as what the bundled tooling actually exposes: a stdio hex-APDU REPL with a READY banner, not an HTTP server and not a framebuffer. Harbor launches one serve process per MC cubby using a per-cubby state prefix, queues one request/response APDU at a time, imposes READY and APDU timeouts, records process metadata, and reaps the process on session/gateway shutdown. The projection page exposes GET serve/status, POST serve/apdu, and an optional WebSocket endpoint. This is a precise and defensible integration claim: the browser controls a live APDU REPL through Harbor; it does not claim a complete remote graphical framebuffer.

## 9.6 Mobile verification and host-specific evidence



*Figure 13 - Mobile/Cubby verification split by evidence class. The fresh Linux brctl proxy shortfall is platform incompatibility of the bundled Windows executable, not an observed protocol failure.*

**Table 12 - Mobile/Cubby verification evidence**

| Verification campaign                 | Pass / fail | What it establishes                                                                                                                            |
|---------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Archived mobile Cubby probe (Windows) | 12 / 0      | MC allocation, registry, desktop direct path, mobile 403 gate, mobile-via-BR attach, HTTP projection, ticket enrichment, API list.             |
| Fresh mobile Cubby probe (Linux)      | 12 / 0      | The same gateway/control-plane behaviors reproduce on the updated source with a locally generated review principal.                            |
| Archived brctl serve proxy (Windows)  | 8 / 0       | Real brctl.exe process reached READY; APDU status request returned 9000...; projection showed ready; HTTP proxy and gate remained operational. |
| Fresh brctl serve proxy (Linux)       | 4 / 3       | Gateway/ticket/projection/gate checks pass; process-specific checks fail because PE32+ brctl.exe cannot execute natively on Linux (EACCES).    |

The archived brctl APDU evidence is especially useful because it goes beyond a page stub: manager\_ensure\_ready observed a live Windows process, manager\_apdu\_status returned a response beginning 9000, the gateway reported serve ready, and POST /serve/apdu returned the APDU response through the Harbor proxy. Fresh Linux review correctly refuses to reinterpret the EACCES host incompatibility as a product failure.

## 9.7 Mobile security and privacy boundaries

**Table 13 - Mobile/Cubby security boundaries**

| Boundary            | Control                                                                      | Residual consideration                                                                                              |
|---------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Mobile identity     | Existing bearer/ticket authentication precedes cubby allocation.             | Any client classification based on headers/User-Agent must remain subordinate to authenticated principal ownership. |
| QN/CS access        | Gateway-enforced mobile_br_cubby_required check.                             | Live MC ownership/session freshness must remain authoritative.                                                      |
| Projection locality | Projection URL binds to Harbor local 127; device_download=false is explicit. | Real remote-device use may require a secure tunnel/proxy architecture if 127 refers to different hosts.             |
| brctl process       | No brctl network listener; Harbor proxies stdio under gateway policy.        | APDU semantics and state files become privileged review surfaces.                                                   |
| Materializer inputs | Pinned/link-resolved inputs and content-addressed output intent.             | External product trees are absent from the archive and require separate provenance controls.                        |

## 9.8 Current mobile limitations

- The browser projection is an operational Cubby/APDU control surface, not a complete Bottle Rocket graphical framebuffer or full host UI.
- The review archive does not include the Bottle Rocket, iOS735\_LCTL, LinearAndroid\_LCTL, or RODEO product trees; only their integration metadata and links are present.
- The included brctl.exe is a Windows PE32+ executable, so fresh process-level brctl verification requires Windows (or an explicitly supported compatibility layer).
- Optional adb/idevice sideload remains outside the primary Harbor entry path and was not used as evidence for Cubby projection correctness.

# 10. R28 Performance Backtest and Ablation

## 10.1 Experimental design

The R28 experiment is the best evidence artifact in the repository for performance claims. It compares a v1 baseline (JSON + base64 data path, process worker) with the RAMWS v2 candidate (binary data frames, byte credits, allocation ledger, thread worker). The archived experiment configuration is marked frozen before data. Each arm uses three independent launches; each launch uses two warmups plus ten warmed runs. Arm order is randomized and confidence intervals are generated with a 10,000-resample cluster bootstrap at the launch level.

Because baseline and candidate differ in both protocol and worker backend, the main arm estimates the effect of the complete architecture change, not a single mechanism. The preserved process-backend ablation is therefore essential for causal interpretation.

*Table 14 - R28 main-arm CPU results*

| Workload              | Baseline mean | Candidate mean | CPU reduction | 95% cluster CI     | H2 gate   |
|-----------------------|---------------|----------------|---------------|--------------------|-----------|
| W1 - Interactive echo | 0.416 ms      | 0.239 ms       | 42.513%       | 40.489% to 44.652% | SUPPORTED |

| Workload            | Baseline mean | Candidate mean | CPU reduction | 95% cluster CI     | H2 gate   |
|---------------------|---------------|----------------|---------------|--------------------|-----------|
| W2 - Bulk output    | 147.56 ms/MiB | 48.00 ms/MiB   | 67.467%       | 66.115% to 68.953% | SUPPORTED |
| W3 - Paste workload | 0.7177 s      | 0.0057 s       | 99.210%       | 99.160% to 99.303% | SUPPORTED |

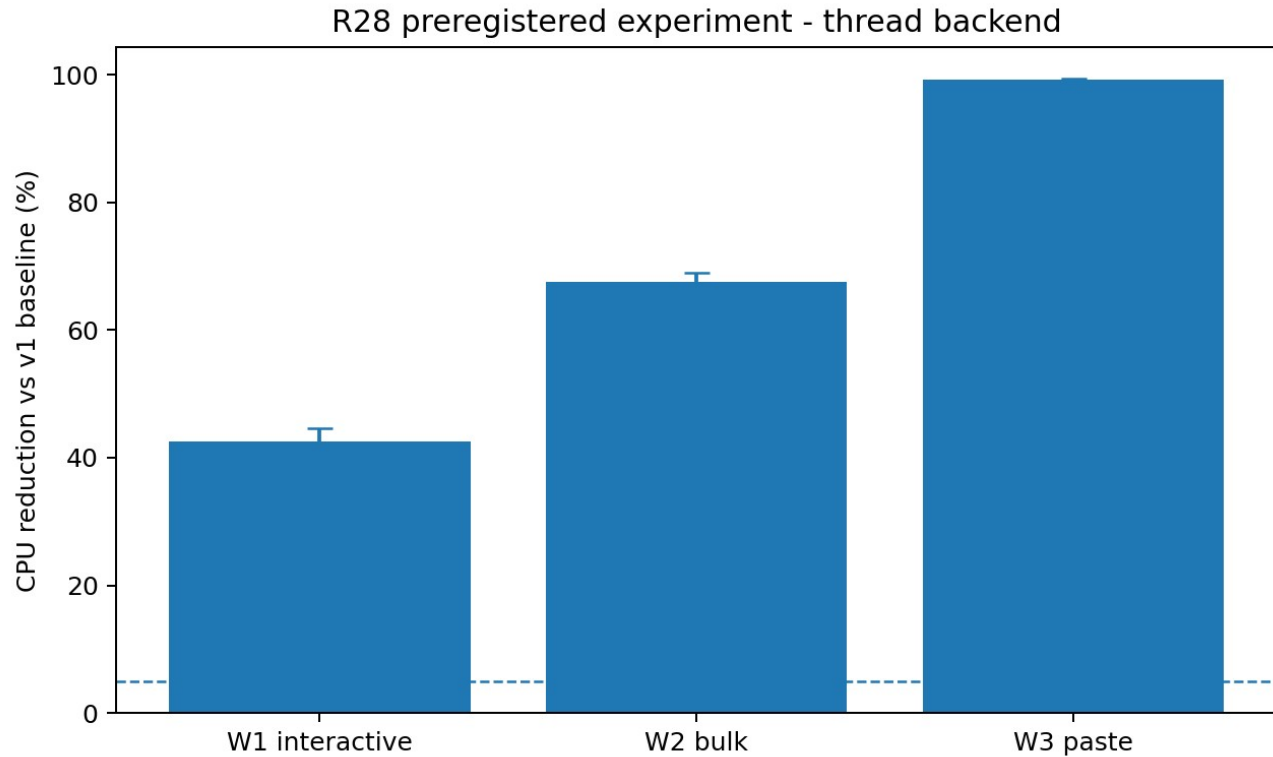


Figure 14 - R28 CPU reduction versus the v1 baseline. Error bars show the archived cluster-bootstrap confidence interval.

## 10.2 Interactive W1 result

The baseline averaged about 0.416 ms of server-tree CPU per echoed keystroke across launches; the candidate averaged about 0.239 ms. The reported reduction is 42.513% with a 95% cluster-bootstrap interval of 40.489% to 44.652%. This is the most important workload for assessing whether the new transport/backend improves the human-interactive path rather than only large transfers.

## 10.3 Bulk W2 result

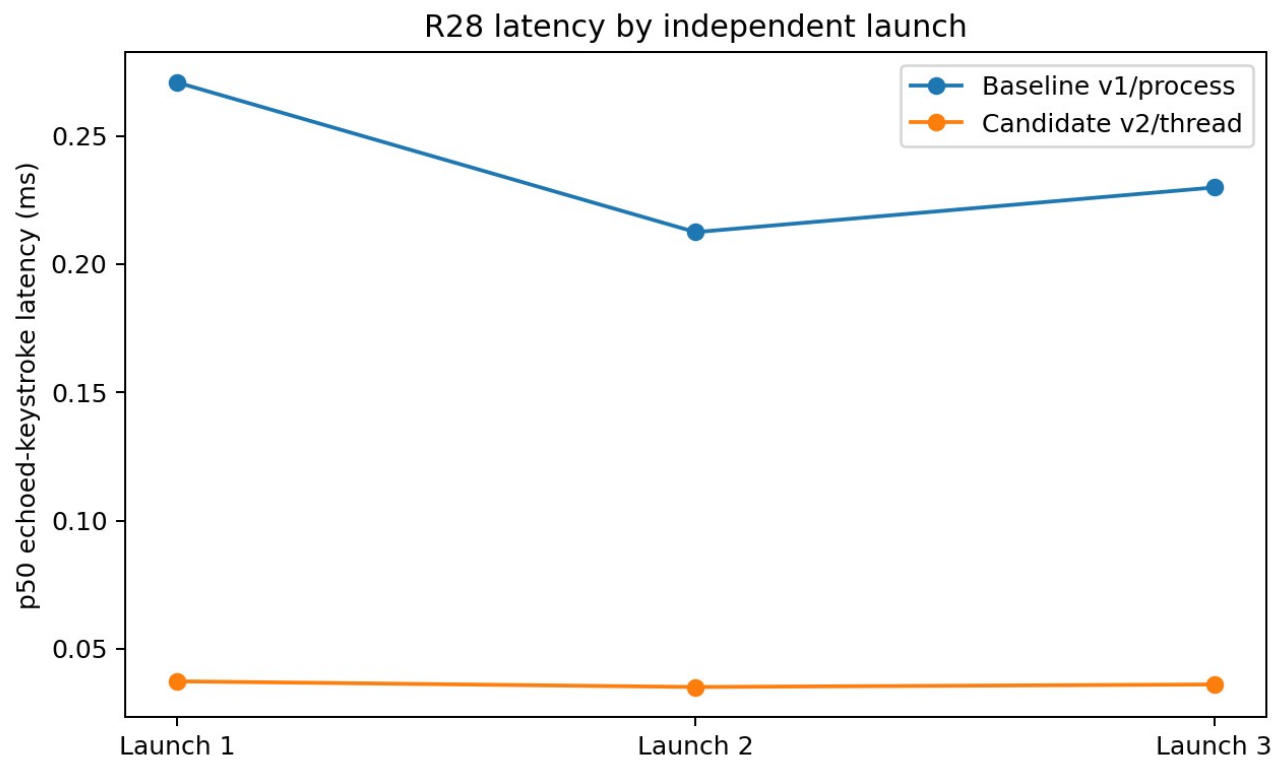
For one MiB of raw terminal output delivered, the baseline averaged about 147.56 ms CPU/MiB while the candidate averaged about 48.00 ms/MiB. The reported reduction is 67.467% (95% CI 66.115% to 68.953%). This is consistent with two architectural changes that remove work from the hot path: binary terminal frames avoid base64 encoding/decoding and worker threads avoid process IPC serialization/copy overhead.

## 10.4 Paste W3 result

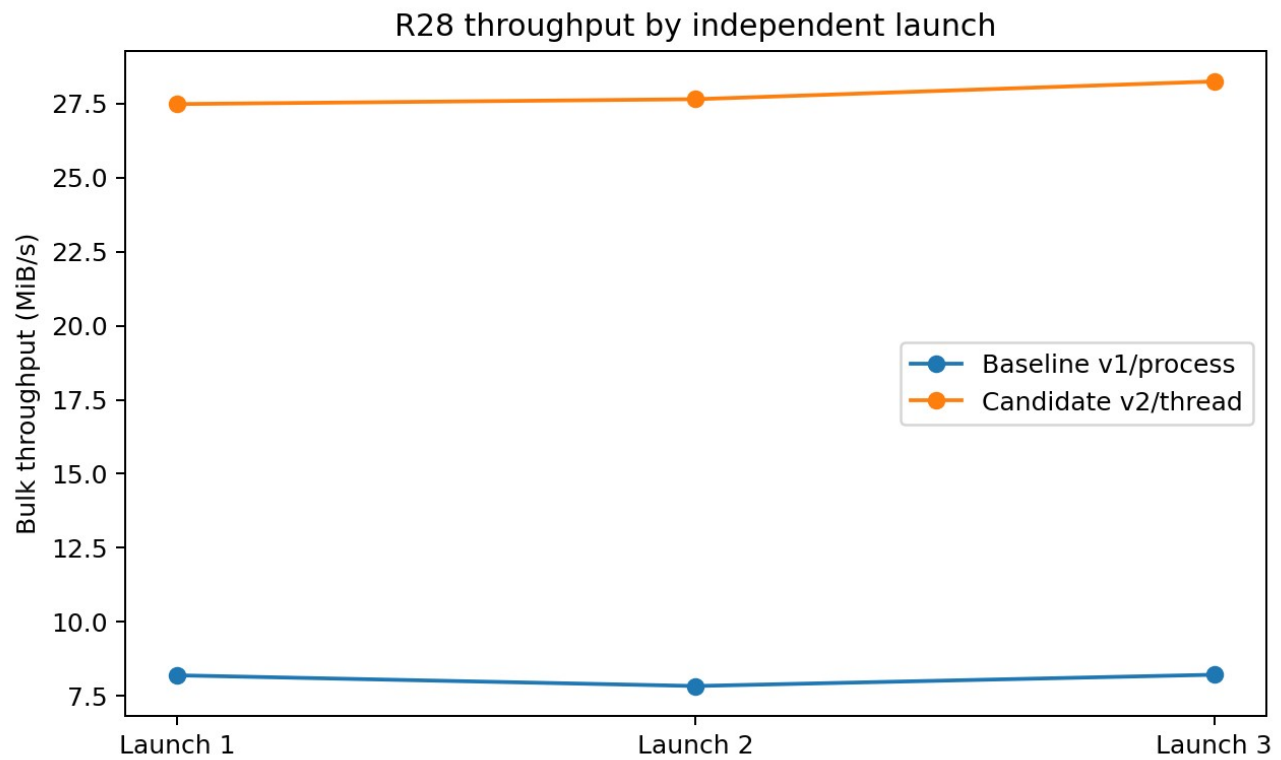
The paste workload shows the largest change: roughly 0.718 s CPU/paste for the baseline versus 0.00567 s for the candidate, a 99.210% reduction (95% CI 99.160% to 99.303%). The archived result also records a

dramatic difference in paste output bytes, which indicates that behavior/protocol changes reduce work amplification. Exact output fidelity is recorded as passing for all included runs.

## 10.5 Latency and throughput



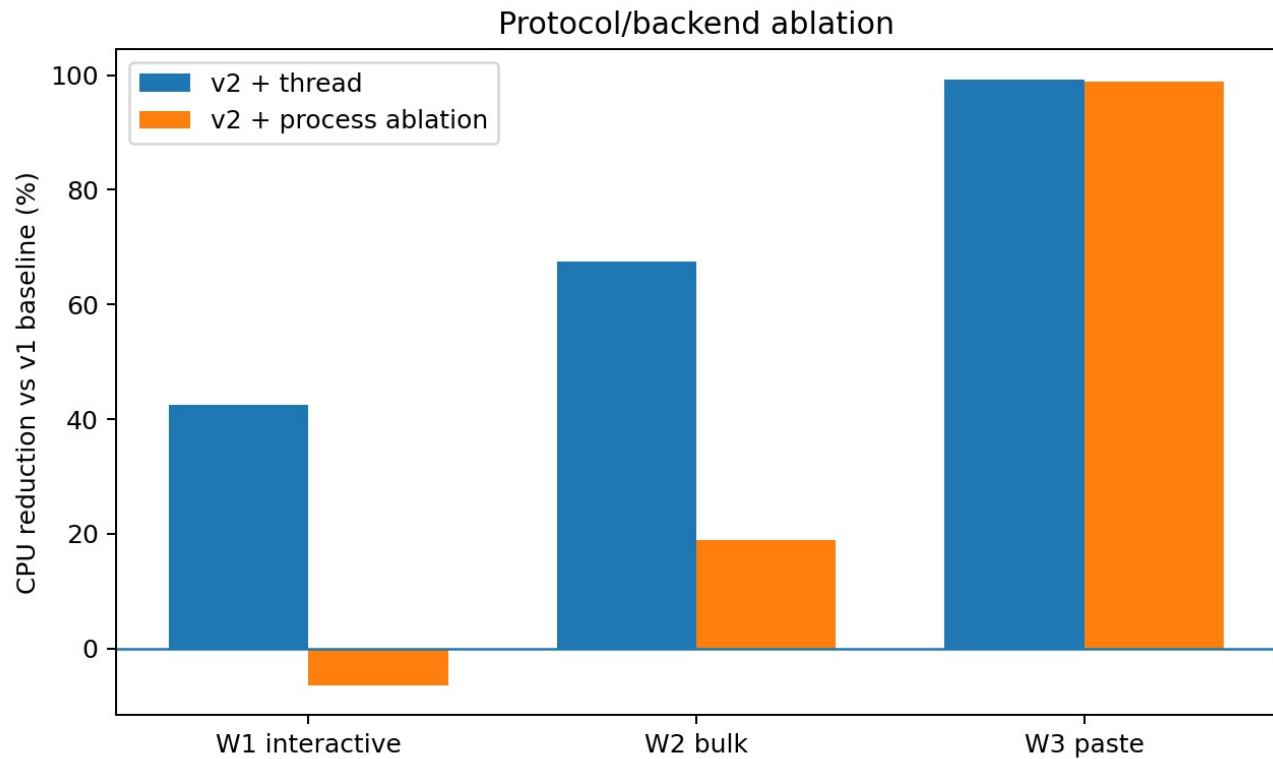
*Figure 15 - R28 p50 loopback latency by independent launch. Candidate values are ~0.035-0.037 ms versus ~0.212-0.271 ms for baseline.*



*Figure 16 - R28 bulk throughput by launch. Candidate mean is ~27.8 MiB/s versus ~8.08 MiB/s for the baseline on the archived 2-vCPU host.*

Across the three launches, baseline p50 latency averages 0.238 ms and candidate p50 latency averages 0.036 ms. Bulk throughput averages 8.08 MiB/s baseline versus 27.80 MiB/s candidate. The experiment records the latency and throughput non-inferiority gates as true; it also records exact fidelity for all runs and no excluded runs.

## 10.6 Process-backend ablation



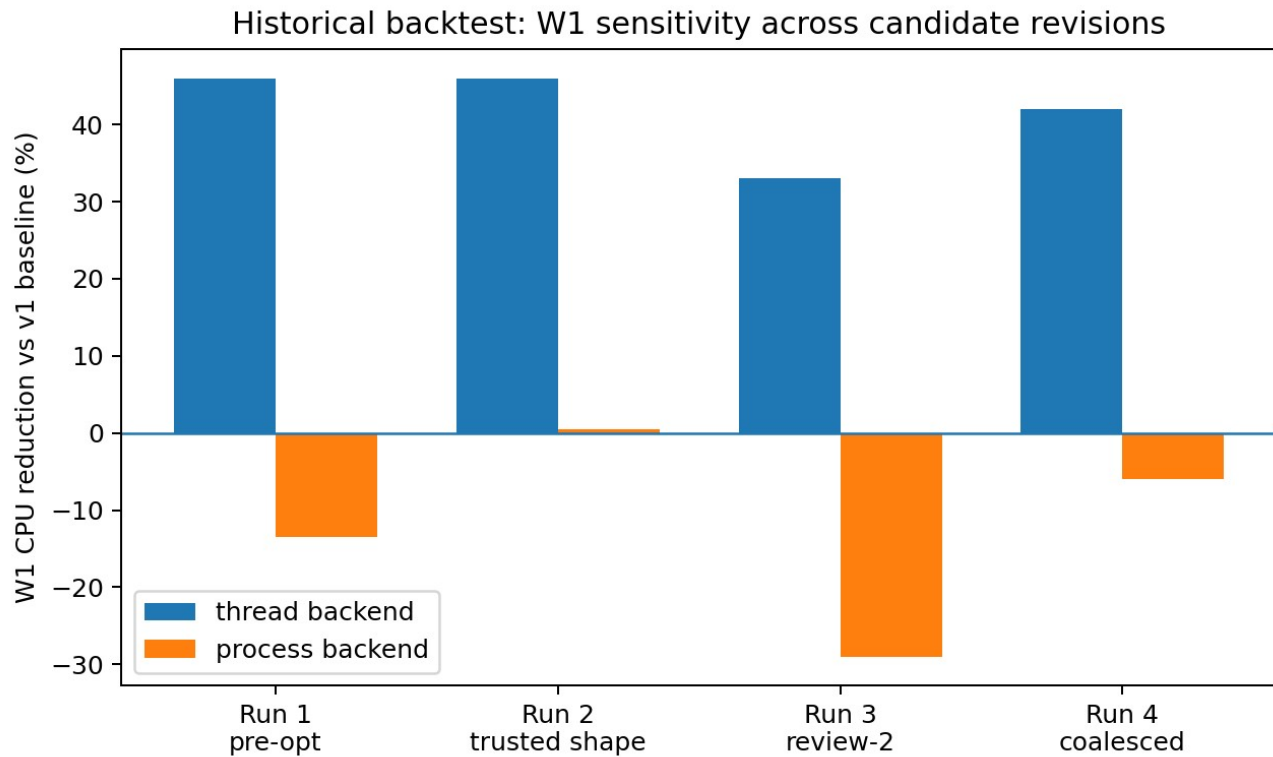
*Figure 17 - Process-backend ablation. Retaining a process worker removes the W1 CPU benefit, while W2 and W3 still retain protocol-related gains.*

*Table 15 - R28 process-backend ablation*

| Workload | CPU reduction, process backend | 95% CI              | Gate          |
|----------|--------------------------------|---------------------|---------------|
| W1       | -6.332%                        | -10.298% to -2.020% | NOT_SUPPORTED |
| W2       | 18.979%                        | 17.630% to 20.249%  | SUPPORTED     |
| W3       | 98.908%                        | 98.853% to 99.000%  | SUPPORTED     |

With the candidate protocol but a process worker, W1 becomes 6.33% worse than the corresponding v1 baseline and its confidence interval remains below zero. W2 still improves by 18.98% and W3 by 98.91%. This pattern is technically informative: the v2 protocol is sufficient for large output/paste gains, but the interactive CPU path appears to depend materially on eliminating process IPC.

## 10.7 Historical revision sensitivity



*Figure 18 - Preserved R28 historical W1 reductions across candidate revisions. The main thread-backend candidate remains positive; the process-backend sensitivity oscillates around or below zero.*

The experiment notes preserve multiple reruns as the candidate evolved. This transparency is valuable for understanding sensitivity, but it also means the implementation itself was not a single immutable object across every historical rerun even though the experiment configuration was frozen. The final archived main result should therefore be read as the selected candidate state under the frozen measurement design, with earlier reruns serving as revision-regression history rather than independent confirmatory studies.

## 10.8 Limits on interpretation

- Single loopback host; no TLS, edge routing, WAN jitter, or multi-host deployment effects.
- Only three launches per arm; bootstrap inference is clustered at a small number of independent launches.
- CPU is measured for the server process tree; client and OS work are excluded.
- Main baseline/candidate arms change both protocol and worker backend; the ablation helps but does not fully factor every mechanism.
- Zero major-fault deltas are interval observations for the measured process scope, not a guarantee that swapping can never occur.

# 11. Qnode Fleet Audit and Bell-Distribution Analysis

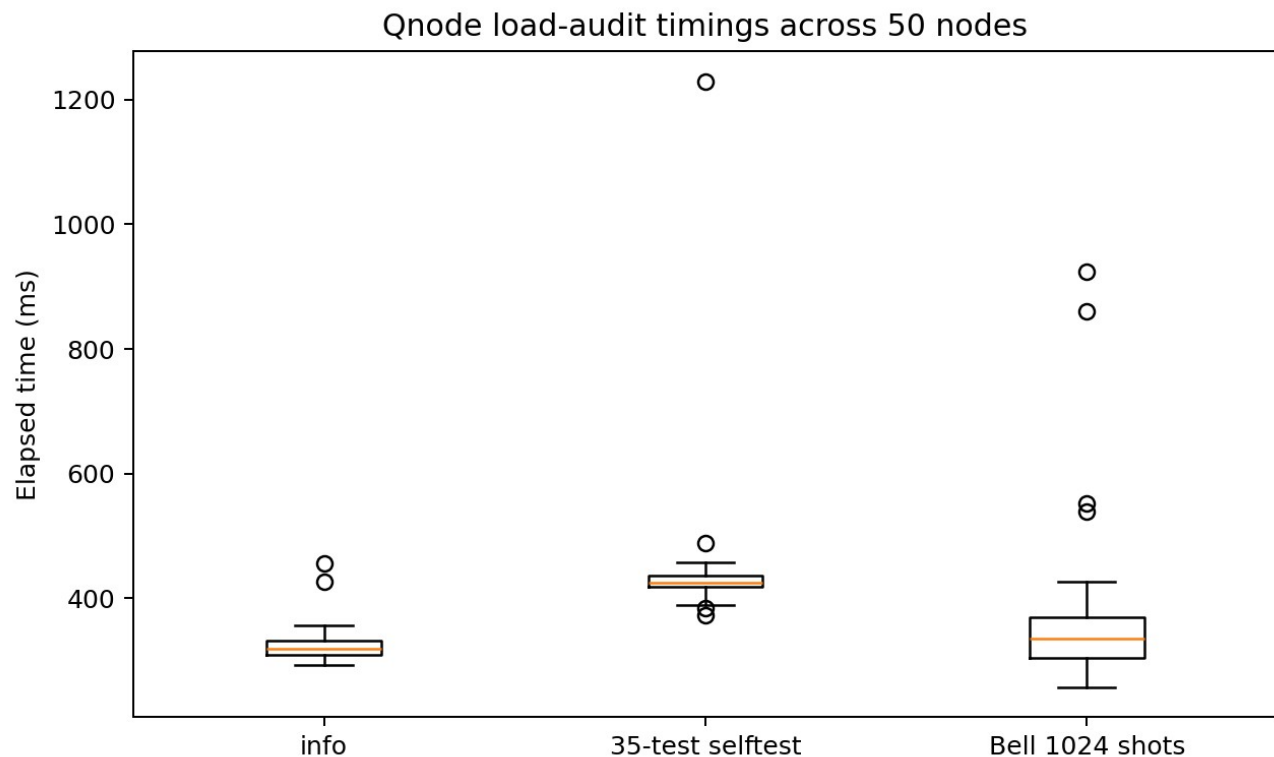
## 11.1 Archived 50-node audit

The fleet audit records all fifty nodes passing inventory, independence, fleet alignment, and a concrete load probe. Per node, the load probe runs `qvm.cli info`, `qvm.cli selftest`, and `qvm.cli run`

examples/bell.json with a statevector Bell circuit and 1,024 shots. The selftest reports 35 tests per node, so the fleet audit represents 1,750 QVM selftests in aggregate.

*Table 16 - Archived Qnode fleet audit*

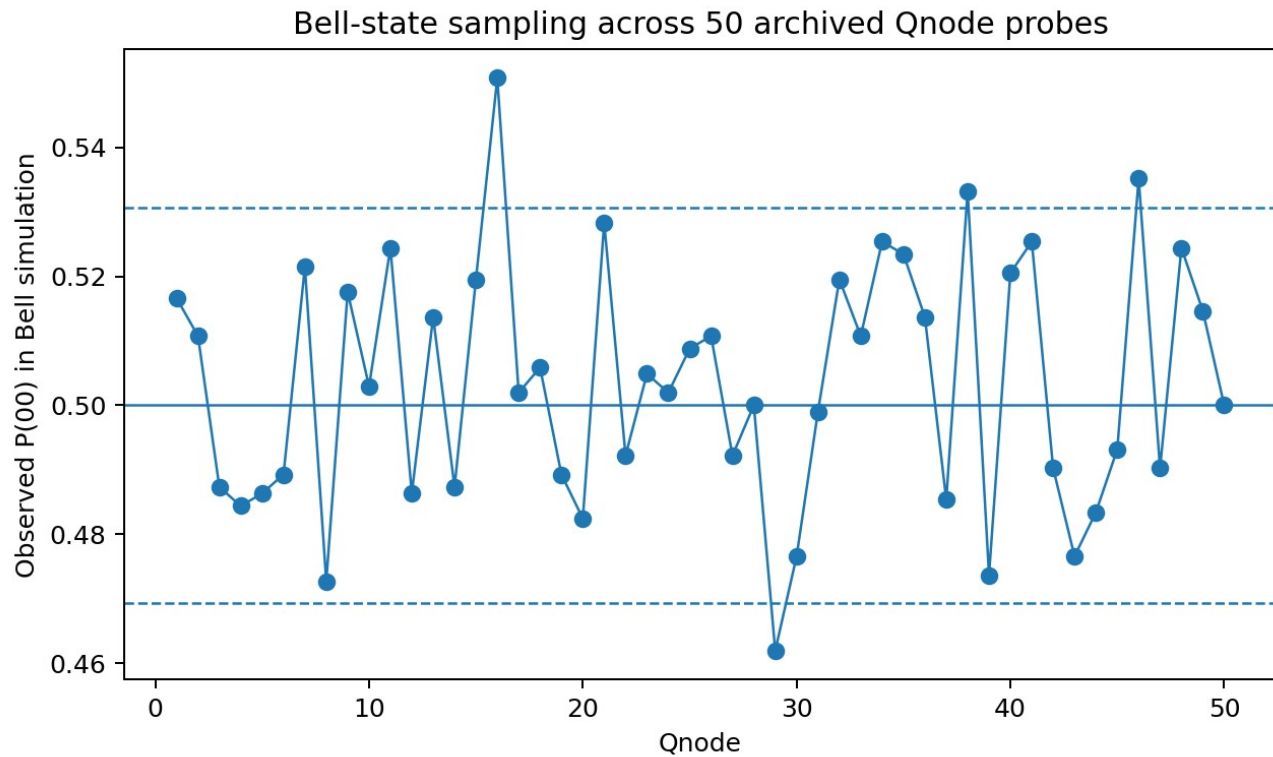
| Audit layer         | Recorded result | Interpretation                                                                                            |
|---------------------|-----------------|-----------------------------------------------------------------------------------------------------------|
| Inventory           | 50/50 pass      | Expected entryptoints and independent-copy characteristics present in the audited full-materialized tree. |
| Independence marker | PASS            | QN-01 runtime marker did not leak into QN-02.                                                             |
| Load probe          | 50/50 pass      | info + 35-test selftest + Bell statevector run succeeded for every node.                                  |
| Fleet alignment     | PASS            | Focus codes qn01..qn50 and full-copies mode aligned with fleet roster.                                    |



*Figure 19 - Archived Qnode command timings across the 50-node load audit. The long selftest maximum reflects a tail outlier rather than a fleet-wide failure.*

The archived timing distributions are modest for a CLI-level simulation audit: info mean 326.2 ms (min 293 / median 319.5 / max 456), selftest mean 442.2 ms (max 1229), and Bell run mean 363.0 ms (max 924). These are not throughput measurements; they are per-node load-probe wall-clock durations in the original audit environment.

## 11.2 Bell simulation aggregate



*Figure 20 - Aggregate Bell-state simulator counts across 50 archived Qnode runs (51,200 shots total).*

Across the fifty 1,024-shot runs, the archived outputs contain 25,770 occurrences of 00 and 25,430 of 11, for  $P(00) = 0.503320$ . A two-category Pearson chi-square test against an exact 50/50 expectation gives chi-square = 2.2578 (1 degree of freedom), with an approximate p-value of 0.133. At conventional levels, that aggregate count does not provide evidence of a systematic bias away from 50/50.

Using an approximate per-node 95% binomial band around 0.5 for  $n=1,024$ , four of the fifty node runs fall outside the individual band. With fifty simultaneous checks, several such excursions are not surprising even when every node samples the same ideal distribution. This calculation is a statistical sanity check on archived simulator output, not evidence of physical quantum hardware or quantum advantage.

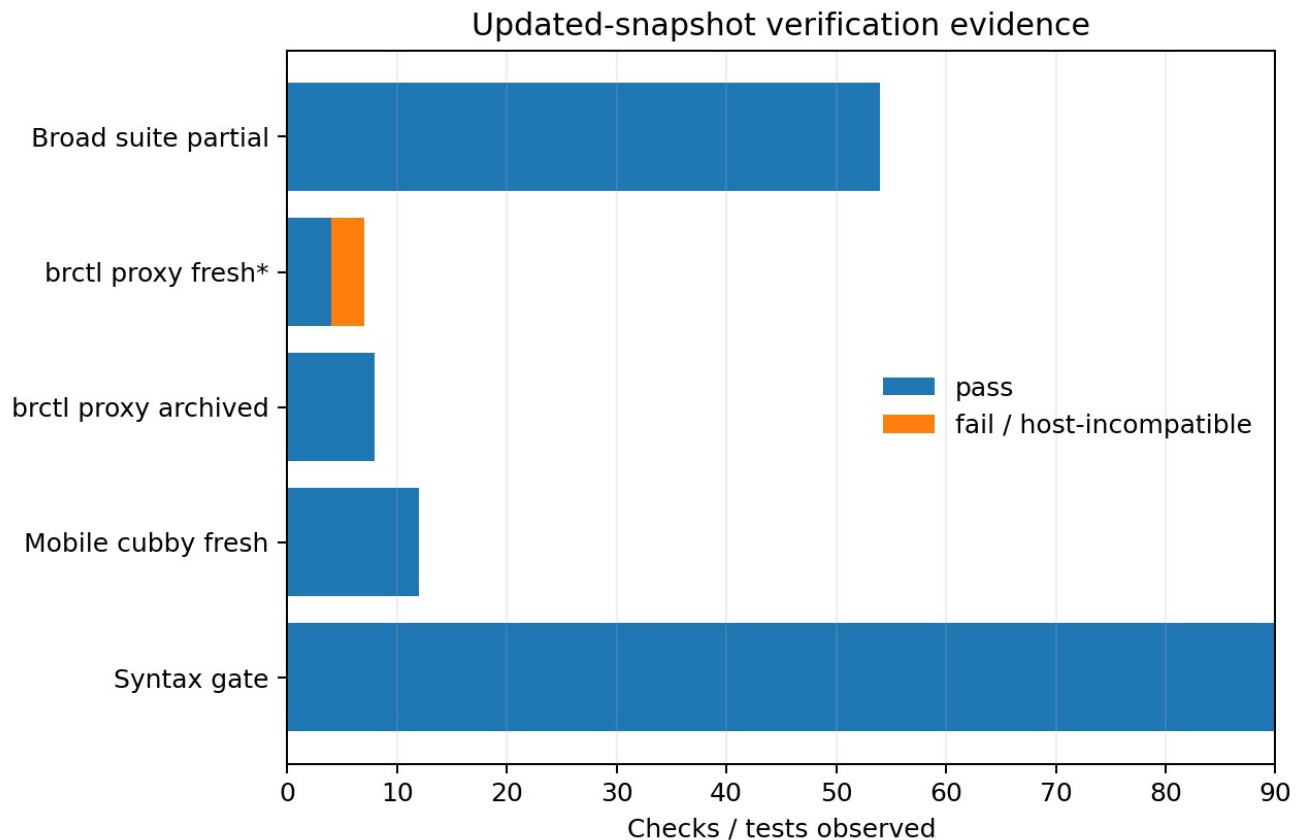
### Quantum-execution scope

The QVM metadata and audit describe local statevector simulation. Physical-QPU execution is not claimed. The Bell analysis validates the consistency of the archived simulator counts with the expected ideal distribution; it does not validate hardware fidelity.

## 12. Fresh Verification Campaign

### 12.1 Updated-snapshot syntax gate

The current bridge-terminal syntax gate completed successfully: 90 JavaScript files checked, 90 passed, 0 failed. The prior snapshot contained 86 checked JavaScript files; the increase corresponds to the newly added Cubby/mobile gateway modules. Syntax success is a precondition, not a behavioral certification.



*Figure 21 - Updated-snapshot verification evidence. The broad npm test run is intentionally shown as partial because the review command window ended before suite completion; no failure had been observed through test 54.*

### 12.2 Fresh mobile Cubby behavioral verification

The mobile-cubby verification script was rerun against the updated tree with a locally generated review principal. All 12 checks passed. The run allocated MC-005 and later MC-006, confirmed that the registry grew, verified desktop-direct QN projection, verified the 403 mobile prerequisite without a BR cubby, verified attach-via-BR for a principal holding the live MC, served the projection page, obtained a mobile ws-ticket containing a cubby id and device\_download=false, and confirmed that /api/cubbies exposed the mobile entries.

### 12.3 Fresh brctl proxy verification: gateway passes, Windows binary is host-limited

A second verifier exercised the brctl serve proxy. On Linux, gateway listen, mobile ticket allocation, projection-page serve UI, and the mobile QN gate passed. Process-specific checks failed because the supplied brctl.exe is a Windows PE32+ x86-64 executable and cannot be spawned natively in the Linux review host (EACCES). The archived Windows evidence separately records 8/8 success including READY

state and a successful APDU response. The report therefore treats the Linux result as an environment limitation, not as a contradiction of the Windows evidence.

## 12.4 Broad bridge-suite partial run

A current npm test invocation was also started against the updated tree. It progressed through numbered test 54 with no observed failures before the review execution window ended. Six DF integration tests were skipped because the external DF containers were not bound in this sandbox. Because the command did not finish, this is intentionally labeled partial evidence; it is not promoted to a current 127/127 full-suite claim.

## 12.5 Archived full-suite and bridge-subset evidence

The repository still preserves bridge-terminal/e/full-suite.tap reporting 127 tests, 127 pass, 0 fail, and the updated documentation corpus includes a 30/30 bridge unit subset covering gateway configuration, protocol decoding, descriptor validation, and the RAM ledger. Those artifacts provide historical regression continuity, but their dates/environments remain distinct from the fresh current-tree runs.

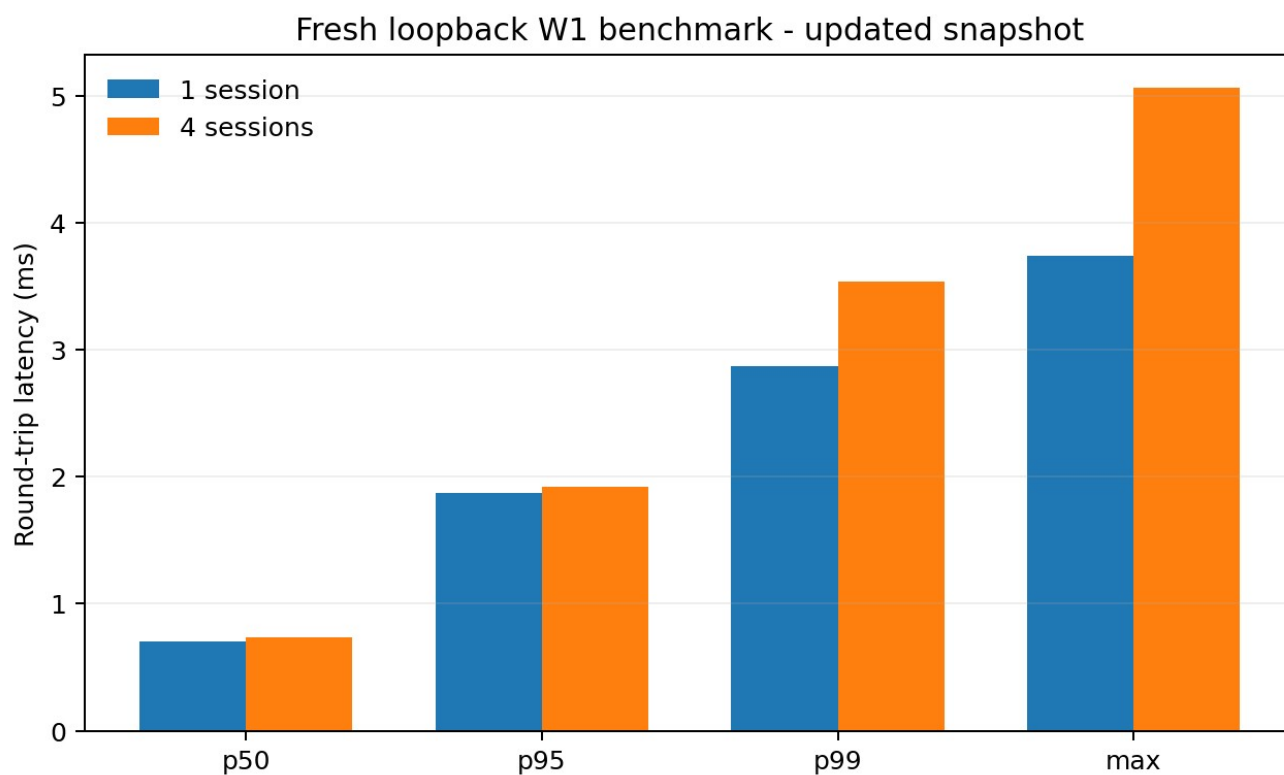
### Current verification conclusion

The updated source clears syntax, the new Cubby gateway path reproduces 12/12 fresh, and the archived Windows brctl proxy demonstrates the process/APDU path 8/8. A clean Windows rerun of the complete bridge suite plus brctl proxy is the appropriate release-grade closure step.

# 13. Load Benchmark and Current Harness Defects

## 13.1 Fresh controlled runs

The load benchmark was run at one and four concurrent sessions because those settings remain compatible with the current default per-principal connection limit. These loopback values are useful as current-tree smoke metrics, not deployment capacity claims and not directly comparable to the R28 host.



*Figure 22 - Fresh W1 echo round-trip latency at 1 and 4 sessions on the review host.*

*Table 17 - Fresh local load-smoke results*

| Metric                            | 1 session                         | 4 sessions                        |
|-----------------------------------|-----------------------------------|-----------------------------------|
| W1 p50 / p95 / p99 / max          | 0.702 / 1.869 / 2.873 / 3.741 ms  | 0.736 / 1.918 / 3.534 / 5.065 ms  |
| W2 raw output / wall / throughput | 6,178,093 B / 1.57 s / 3.74 MiB/s | 6,178,093 B / 1.64 s / 3.59 MiB/s |
| W3 session-open p50 / max         | 165 / 165 ms                      | 134 / 135 ms                      |
| Gateway RSS before -> sessions    | 109.2 -> 118.7 MiB                | 107.5 -> 148.2 MiB                |

## 13.2 Default eight-session contradiction

The benchmark defaults to `BENCH_SESSIONS=8` and raises `maxSessions` and `maxSessionsPerPrincipal` to match that number, but it does not raise `maxConnectionsPerPrincipal`, whose default remains four. The benchmark creates one client connection per session under the same principal. Consequently the fifth connection can be rejected before the intended eight-session scenario is reached. The default benchmark therefore violates its own connection policy unless the connection cap is explicitly adjusted.

## 13.3 Stale wire-expansion annotation

The W2 output JSON still emits `wireExpansion=1.333` with a note attributing the expansion to base64 in JSON text frames. That description matches the v1 baseline, not the current RAMWS v2 binary data path. The measured raw bytes and wall time remain usable, but the wire-expansion field/note should be removed or recomputed from actual WebSocket bytes for v2.

### 13.4 Worker RSS telemetry blind spot

The benchmark attempts to sample `x.worker.pid`, which existed for process workers. A Node Worker thread has no child process PID, so the current thread-backend run records zero worker RSS samples and null min/max/mean. Thread memory should instead be measured through the Node process RSS plus ledger/heap statistics or worker performance/resource APIs appropriate to the implementation. Reporting null is preferable to inventing a per-worker number, but the benchmark should make the backend distinction explicit.

## 14. Release Integrity and Provenance Drift

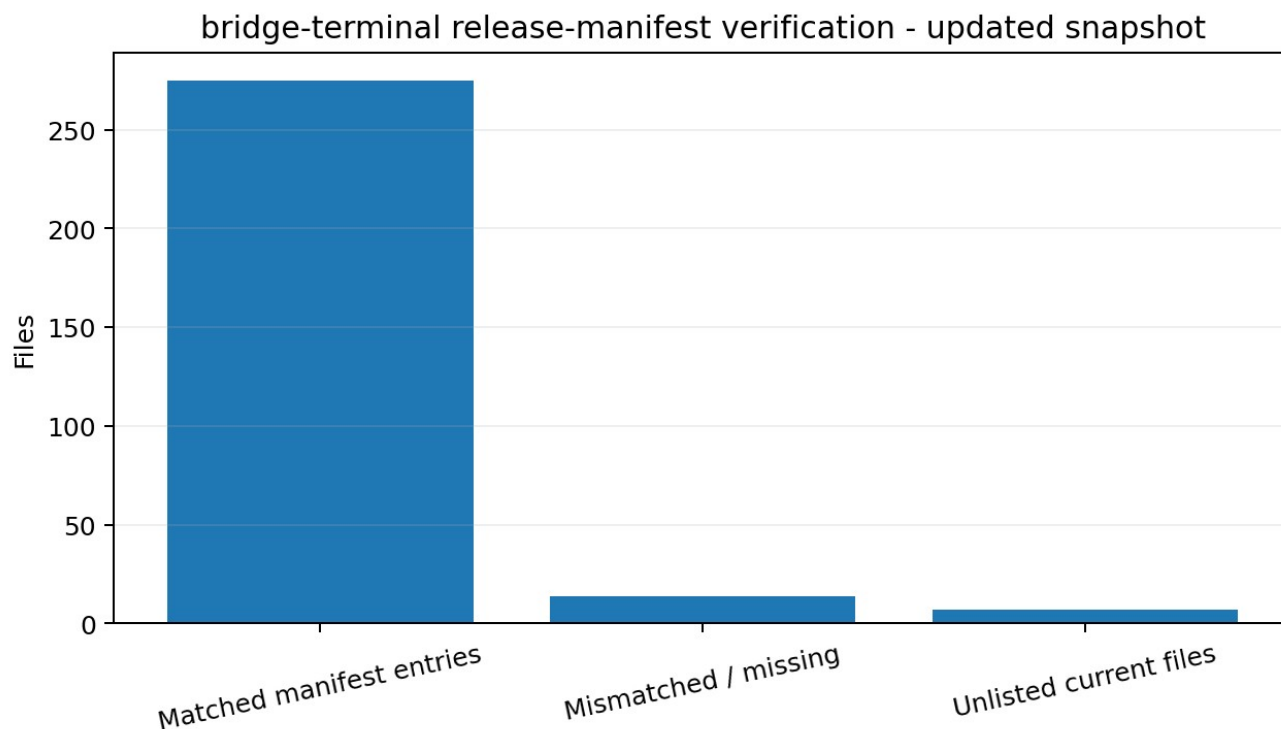


Figure 23 - Current bridge-terminal release-manifest verification against the uploaded working tree.

A fresh `npm run verify` checks the current bridge tree against its release manifest. The verification reports 289 manifest files examined, with 14 mismatched or missing entries and seven current files not listed. Equivalently, 275 manifest entries still match. Four of the seven unlisted files are the newly added Cubby/mobile gateway modules (`brctl-serve-manager.js`, `cubby-access-gate.js`, `cubby-projection.js`, `mobile-auth-hook.js`); the other three are the previously integrated Qnode `command/locator/runner` modules. `gateway/server.js` is now also mismatched because the mobile/Cubby routes were added after the sealed manifest.

Table 18 - Manifest drift categories

| Finding  | Examples                                                                                               | Meaning                                        |
|----------|--------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Mismatch | START.cmd, gateway/connection.js, gateway/server.js, src/main/main.js, kernel.js, tools/start-local.js | Current bytes differ from the sealed manifest. |

| Finding  | Examples                                                     | Meaning                                                            |
|----------|--------------------------------------------------------------|--------------------------------------------------------------------|
| Missing  | Several e/R28 *.log files                                    | Manifest references evidence logs absent from this snapshot.       |
| Unlisted | 4 Cubby/mobile gateway modules + 3 Qnode integration modules | Current integration files were added after the manifest inventory. |

The most plausible interpretation is unreleased integration drift rather than random corruption: the unlisted files and several changed entypoints align with Harbor/Qnode integration work. Nevertheless, a release manifest is only useful if it describes the delivered state. The correct remediation is to review the intentional changes, regenerate/reseal the manifest for a named Harbor release, retain the old manifest with its historical build if necessary, and rerun verification until the current release is internally consistent.

#### Provenance rule

Do not convert the current verify failure into a PASS by weakening verification. First decide which changes are intended, then create a new sealed release manifest that exactly describes the intended tree.

## 15. Containership / Edge Evidence

### 15.1 Current status commands

Fresh read-only containership status commands identify UC-2.8.0 and report that pinned DF\_Fabric / DF node archive digests match their expected SHA-256 values. The fresh environment has Linux 6.18.44 and Python 3.13.5 with common build tools visible, but containership engines are not extracted and the hull is not installed in this sandbox. A full verify --quick --no-hull attempt exceeded the command window and was terminated; this report therefore does not claim a clean fresh containership verification.

15.2 Archived edge regression evidence

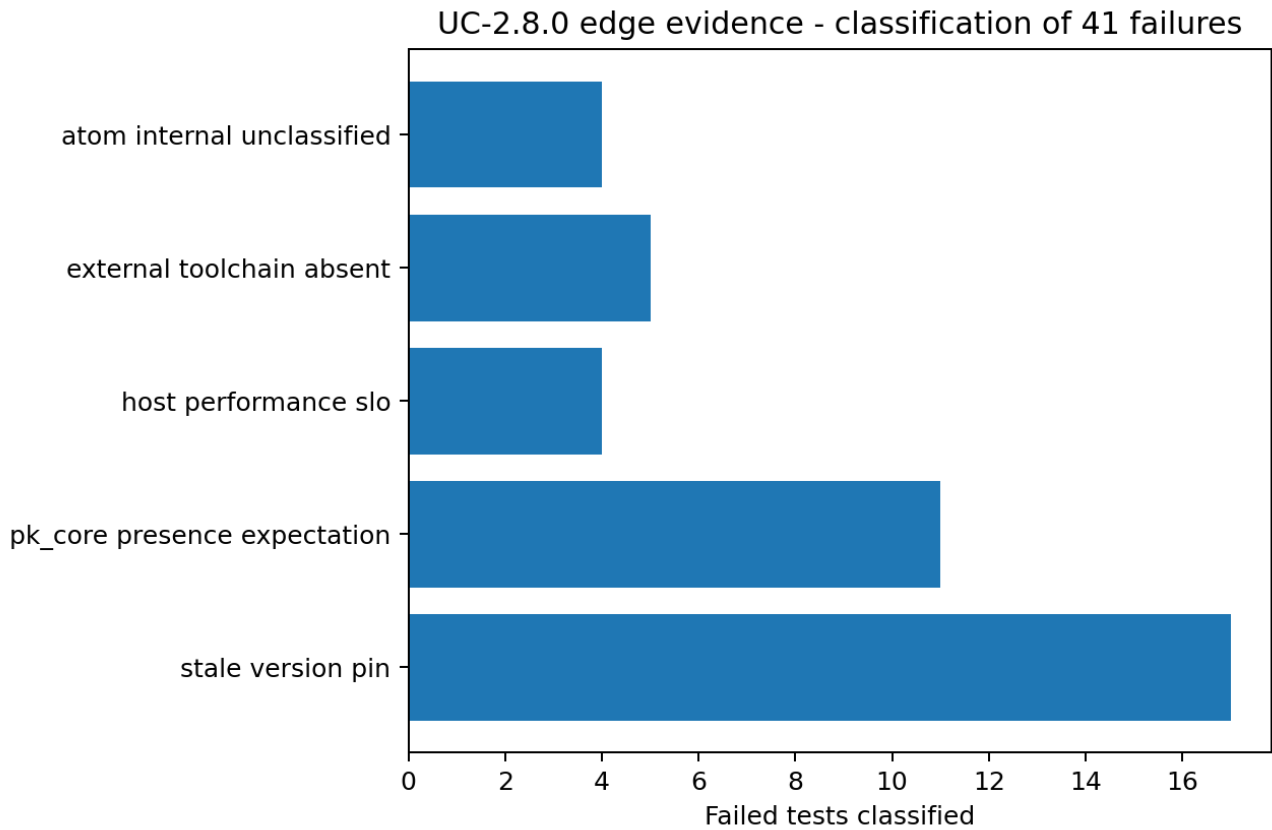


Figure 24 - Classification of the 41 failures in the archived UC-2.8.0 edge evidence. The largest classes are stale version pins and pk\_core presence expectations.

The archived edge evidence records 9,833 passed tests, 41 failed, 46 skipped, and 0 errors. At the suite level it records 59 SUITE\_GREEN and 27 SUITE\_RED verdicts across the 86 canonical atom suites. Importantly, the failure evidence is classified instead of collapsed into a single red count.

Table 19 - Archived edge failure classification

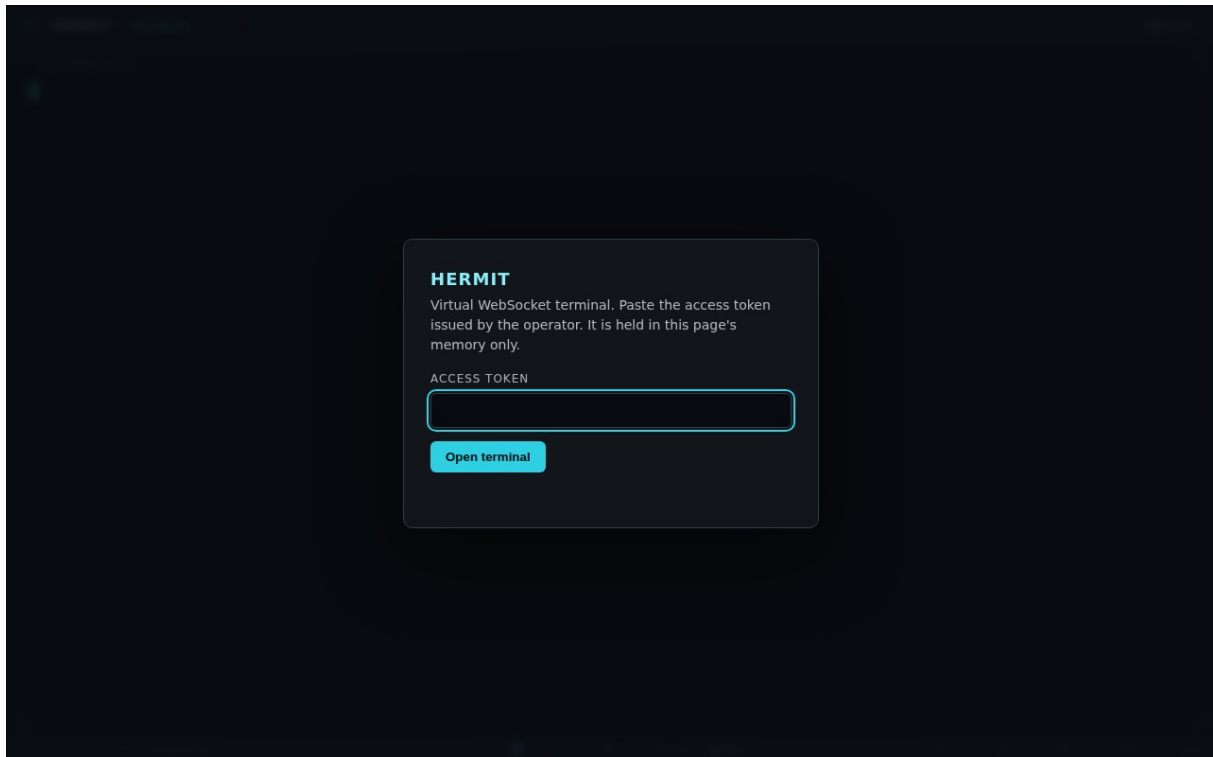
| Failure classification       | Count | Engineering reading                                                                                  |
|------------------------------|-------|------------------------------------------------------------------------------------------------------|
| stale-version-pin            | 17    | Expected version metadata lags the supplied/current component version.                               |
| pk_core-presence-expectation | 11    | Test expectation disagrees with the supplied pk_core layout/version.                                 |
| external-toolchain-absent    | 5     | Host lacks a required external build/toolchain dependency.                                           |
| host-performance-slo         | 4     | Host performance did not meet a prescribed SLO.                                                      |
| atom-internal-unclassified   | 4     | Needs component-level investigation; not yet normalized into a known environmental/provenance class. |

## 15.3 Platform/master-workflow status

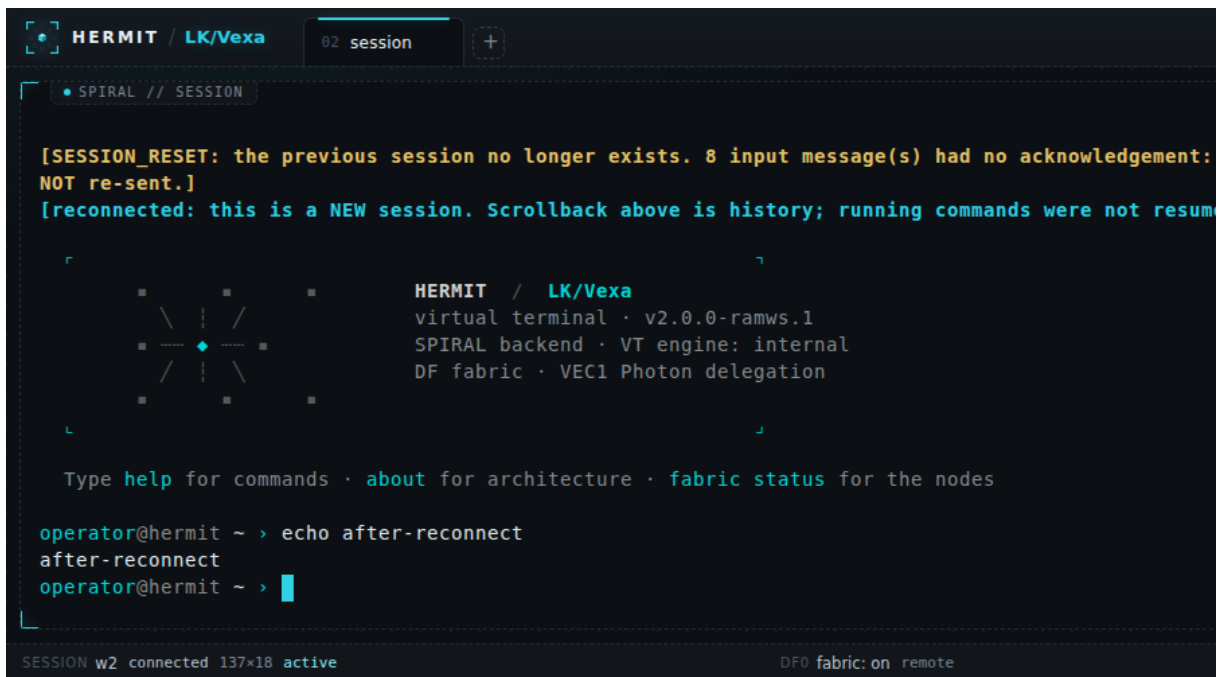
The current platform status exposes a much larger orchestration backlog: a 375,000-task master workflow with 265,000 OPEN and 110,000 BLOCKED tasks, and 150 components distributed among IN\_PROGRESS, OPEN, and BLOCKED states. The edge test JSON explicitly notes that atom tests do not automatically promote PK/master workflow tasks. Therefore the 9,833 passing edge tests should not be translated into a blanket "production ready" verdict for the whole containership platform.

# 16. Reliability, Recovery, and Operational Characteristics

## 16.1 Reconnect without replay



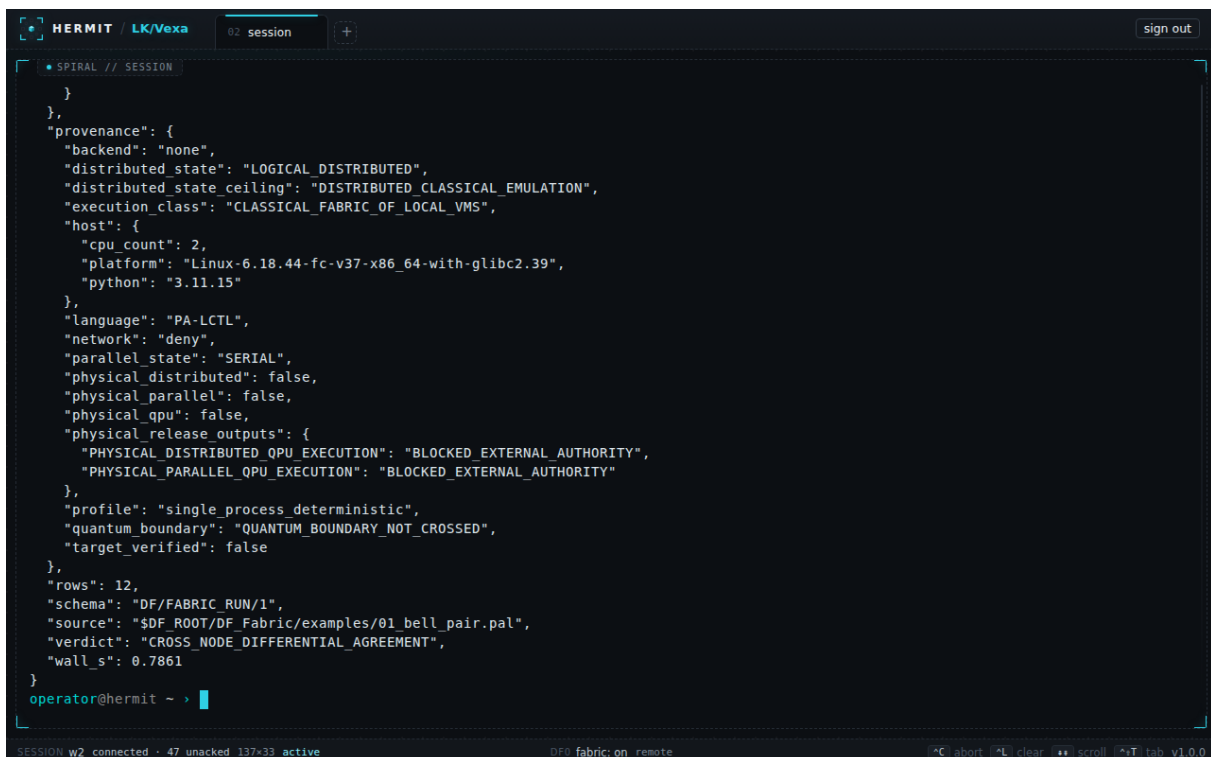
*Figure 25 - Archived Harbor sign-in UI evidence. The local operator token is supplied to the loopback gateway rather than embedded in the public repository.*



*Figure 26 - Archived reconnect evidence. SESSION\_RESET opens a fresh session and explicitly does not resend eight pending unacknowledged inputs.*

The reconnect behavior is consistent with LOCAL\_VOLATILE semantics: the client retains enough protocol state to know that a reset occurred, but the new session is not treated as durable continuation. That reduces accidental duplicate command execution at the price of requiring the operator/application to decide whether any interrupted work must be reissued.

## 16.2 Fabric run evidence



*Figure 27 - Archived UI evidence of a fabric run from the terminal. The output identifies logical distributed execution and no physical QPU.*

The fabric screenshot is useful primarily as integration evidence: terminal command, fabric runner, and UI output form a single visible workflow. The displayed result should not be read as proof of distributed physical hardware; the output itself identifies logical/discrete execution characteristics and no physical QPU.

### 16.3 Exactly-once exit and bounded failure

Reliability in RAMWS is biased toward determinism and bounded failure rather than transparent durability. Session exit is exactly once, output sequencing is contiguous, transport and worker resources are admitted before creation, and slow consumers eventually hit bounded timeout/quota behavior. This is appropriate for a local technical terminal where duplicate command replay can be more dangerous than explicit session loss.

## 17. Engineering Findings and Remediation Priorities

*Table 20 - Recommended remediation backlog*

| Priority                           | Finding                                                                                         | Recommended action                                                                                                          |
|------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| P0 - release integrity             | Bridge release manifest does not describe the current Harbor-integrated tree.                   | Review intended changes, reseal a named release, regenerate manifest, require verify=PASS before distribution.              |
| P0 - benchmark correctness         | Default load benchmark requests 8 same-principal connections while cap remains 4.               | Set maxConnectionsPerPrincipal >= BENCH_SESSIONS in benchmark fixture or multiplex sessions over intended connection model. |
| P1 - test robustness               | Gateway input.ack assertion is contention-sensitive.                                            | Use condition-based waiting / deterministic fake clock where possible; isolate timing-sensitive E2E from high-load groups.  |
| P1 - test oracle                   | RAM heap-cap test only recognizes "capture limit" or "quota" diagnostics.                       | Assert the invariant (bounded refusal / no runaway allocation) or accept all canonical limit error codes/messages.          |
| P1 - telemetry                     | Thread-backend benchmark samples a nonexistent child-worker PID.                                | Add backend-aware memory telemetry using process RSS + ledger + worker heap/resource statistics.                            |
| P1 - protocol benchmark metadata   | v2 benchmark still labels wire expansion as base64/JSON 1.333x.                                 | Measure actual binary WebSocket wire bytes or remove stale field.                                                           |
| P1 - portability                   | QVM full copies and JA21 portable are absent; absolute PRODUCT_LINK paths are machine-specific. | Publish dependency bundle hashes + acquisition/materialization workflow or optional self-contained test fixture.            |
| P2 - containership clean-room gate | Full containership verify was not reproduced in this review environment.                        | Run on a clean short-path Windows/Linux CI host with required hull/toolchains and persist immutable logs.                   |
| P2 - experiment independence       | R28 has only 3 independent launches per arm.                                                    | For publication-grade inference, increase launch clusters and repeat on multiple hosts/CPU classes.                         |

## 17.1 What is already strong

- Protocol design distinguishes control and data planes and gives input/output sequence progression explicit semantics.
- Memory is modeled with admission, ownership, references, and release rather than relying only on garbage collection.
- Backpressure closes the loop at client consumption, not merely server write completion.
- R28 preserves baseline, candidate, ablation, fidelity gates, launch-level measurements, and caveats.
- Archived edge/Qnode evidence exposes failures and environmental constraints instead of reporting an undifferentiated success badge.
- The terminal shell uses an in-memory VFS and explicit privileged adapters, which makes the host-capability surface reviewable.

## 17.2 What should not be overclaimed

- The current uploaded ZIP is not a fully self-contained reproduction of JA21 or the full QVM fleet.
- Archived 127/127 bridge results are historical evidence, not a substitute for resealing and rerunning the current integrated tree.
- R28 loopback numbers are not WAN, TLS, multi-tenant, or deployment-capacity measurements.
- The Qnode Bell results are statevector-simulator sanity checks, not physical-QPU validation.
- The Node worker thread is not an OS security boundary.
- Containership edge test volume does not imply master-workflow completion or a production-go decision.

# 18. Reproduction Guide

## 18.1 Bridge terminal

From bridge-terminal on a host with Node  $\geq 20$  (the package metadata declares  $\geq 20$  for this snapshot):

```
npm install
npm run check
npm test
npm run verify
BENCH_SESSIONS=1 node tests/load/bench.js
BENCH_SESSIONS=4 node tests/load/bench.js
```

For a release gate, correct the known load-fixture and test-oracle issues first, then run the full test suite in a controlled serial/grouped configuration and preserve TAP/JSON output with runtime/host metadata.

## 18.2 Harbor local launcher (intended Windows workflow)

```
START_HARBOR.cmd
START_HARBOR.cmd --no-browser
START_HARBOR.cmd --no-fabric
START_HARBOR.cmd --port 10000
START_HARBOR.cmd --reuse
```

A portable reproduction additionally requires the QVM seed/product and JA21 portable referenced by the link/materialization scripts. The current ZIP does not carry those product payloads.

## 18.3 Mobile Cubby verification

```
cd Harbor-Bridge-Terminal-main
REM Windows: run with a local ACCESS_TOKEN/principals file
node docs\verification\mobile-cubby\_run_verify.js
node docs\verification\brctl-serve-proxy\_run_verify.js
```

The first script validates the portable Node gateway/Cubby contract. The second additionally requires an executable brctl for full READY/APDU closure; the bundled executable is Windows x86-64. Preserve the pristine ZIP separately because verification allocates MC-\* entries and writes session/evidence files into the working tree.

## 18.4 Qnode audit (when full copies are materialized)

```
scripts\materialize-qnode-copies.cmd
scripts\audit-qnode-load.cmd
set QNODE_AUDIT_PARALLEL=5
scripts\audit-qnode-load.cmd
```

## 18.5 Containership

```
cd containership
python uc.py status
python uc.py platform status
python uc.py edge status
python uc.py verify --quick --no-hull
```

Use a clean copy for verification because some containership commands create run/evidence state. Preserve the pristine archive separately from the mutable verification workspace.

## Appendix A. RAMWS v2 Protocol Reference

### A.1 Control envelope

```
{
  "v": 2,
  "type": "<control-type>",
  "payload": { ... },
  "rid": "<optional request id>",
  "sid": "<session id when required>",
  "epoch": "<session/server epoch when required>"
}
```

The exact allowed/required fields are type-specific. The critical review point is that control decoding is bounded and strict before business logic runs.

### A.2 Binary data frame

*Table 21 - Binary frame layout*

| Offset | Width   | Meaning                                    |
|--------|---------|--------------------------------------------|
| 0      | 1 byte  | kind: 1=input, 2=output                    |
| 1..6   | 6 bytes | unsigned 48-bit big-endian sequence number |
| 7..end | bounded | raw terminal bytes                         |

### A.3 Core limits and invariants

*Table 22 - Selected RAMWS v2 invariants*

| Invariant / limit           | Value / behavior                                                  |
|-----------------------------|-------------------------------------------------------------------|
| Control message cap         | 65,536 bytes                                                      |
| Input data cap              | 8,192 bytes per message                                           |
| Output data cap             | 16,384 bytes per message                                          |
| Outstanding output messages | Bounded; documented maximum 1,024                                 |
| Credit behavior             | Cumulative, monotone, bounded by one window                       |
| Slow-consumer behavior      | Output pauses; prolonged pause reaches bounded quota/timeout exit |
| Output sequence             | Contiguous from 1                                                 |
| Session exit                | Exactly one session.exit                                          |
| Durability                  | LOCAL_VOLATILE; no replay guarantee                               |

| Invariant / limit     | Value / behavior                                                |
|-----------------------|-----------------------------------------------------------------|
| Data frame session ID | Omitted; data belongs to authenticated/bound connection session |

## Appendix B. Evidence Inventory and Limitations

### B.1 Primary repository evidence paths

README.md  
 START\_HARBOR.cmd  
 bridge-terminal/README.md  
 bridge-terminal/package.json  
 bridge-terminal/docs/PROTOCOL\_V2.md  
 bridge-terminal/docs/RAMWS.md  
 bridge-terminal/docs/SECURITY.md  
 bridge-terminal/docs/EXPERIMENT.md  
 bridge-terminal/docs/vws200/RESOURCE\_MODEL.md  
 bridge-terminal/e/R28/result.json  
 bridge-terminal/e/R28/result.process-backend.json  
 bridge-terminal/e/full-suite.tap  
 bridge-terminal/e/I055/01-signin.png  
 bridge-terminal/e/I055/02-fabric-run.png  
 bridge-terminal/e/I055/03-reconnected.png  
 fleet/QNODE\_LOAD\_AUDIT.md  
 fleet/QNODE\_LOAD\_AUDIT.json  
 containership/README\_START\_HERE.md  
 containership/edge/evidence/EDGE\_TEST\_RESULTS.json  
 qvm/PRODUCT\_LINK.txt  
 optical-desktop/PRODUCT\_LINK.txt  
 docs/MOBILE\_CUBBY\_PROJECTION.md  
 docs/verification/mobile-cubby/EVIDENCE.json  
 docs/verification/brctl-serve-proxy/EVIDENCE.json  
 fleet/CUBBIES.json  
 fleet/MOBILE\_PLATFORM.json  
 mobile-platform/compiler/cubbies/registry.js  
 bridge-terminal/gateway/cubby-access-gate.js  
 bridge-terminal/gateway/cubby-projection.js  
 bridge-terminal/gateway/mobile-auth-hook.js  
 bridge-terminal/gateway/brctl-serve-manager.js

### B.2 Fresh evidence files produced during this review

hbt\_v2\_fulltest.log  
 hbt\_v2\_bench\_1.json

hbt\_v2\_bench\_4.json

hbt\_v2\_verify.log

docs/verification/mobile-cubby/EVIDENCE.json (fresh working copy)

docs/verification/brctl-serve-proxy/EVIDENCE.json (fresh working copy)

## B.3 Limitations

- No external network benchmark was performed; all bridge measurements are loopback/local.
- The review environment is Linux, while the intended one-click Harbor integration includes Windows CMD/PowerShell and WPF JA21 components.
- Full QVM products are absent from the ZIP, so Qnode execution evidence is archival.
- JA21 portable is absent from the ZIP, so browser launch evidence is code + archived screenshot/documentation rather than fresh execution.
- Bottle Rocket, iOS735\_LCTL, LinearAndroid\_LCTL, and RODEO product trees are absent from the ZIP; mobile gateway/Cubby integration is present, but full VM materialization depends on those external links.
- The bundled brctl.exe is Windows-only. Fresh Linux review can validate gateway/projection/gating but not execute the real brctl process; archived Windows evidence covers that path.
- DF containers were not bound in the fresh environment; corresponding integration tests skip rather than fabricate a target.
- The containership full verify did not finish within the review command window, and the working extraction became mutable; pristine inventory therefore comes from the ZIP central directory.
- Statistical calculations on Qnode Bell counts assume the archived counts are correctly parsed from the preserved audit JSON; they are sanity checks, not hardware characterization.

## B.4 Final technical assessment

Harbor Bridge Terminal is best understood as a local, resource-bounded control workstation whose center of gravity is now the combination of RAMWS/SPIRAL and the Cubby control plane. The bridge supplies the terminal protocol, sandbox-oriented shell, explicit host adapters, sequence/credit semantics, and allocation ledger; Cubby integration extends that discipline to execution-slot identity, mobile admission, projection, and per-session Bottle Rocket control. The archived R28 study still gives the transport architecture a credible quantitative performance history, while the new 12/12 Cubby reproduction and archived 8/8 brctl proxy evidence show that the mobile path is implemented as enforceable gateway behavior rather than documentation-only intent.

The highest-value next step is to turn the present integrated working tree into a reproducible Cubby-capable release candidate: materialize and pin the external mobile products on a clean Windows host, rerun the 12-step Cubby and 8-step brctl proxy probes, run the complete bridge and containership gates to completion, correct the remaining load-harness telemetry assumptions, reseal the manifest so the four Cubby modules and Qnode modules are first-class release files, and preserve the resulting evidence as a new immutable release package. That would align release provenance with the sophistication of the updated architecture.